In [1]:
```
import glob
import os
import pandas as pd
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, c
pd.set_option('display.max_columns', None)

events = pd.read_parquet("../data/gdelt/events/6_final/events_dataset_v2.par
events = events[events['period'] <= '202406']
```

In [2]:
```
print(len(events.index))
print(events.columns)
events.tail(3)
```

```
71293
Index(['ADMIN0', 'ADMIN1', 'ADMIN2', 'CS_score', 'period', 'SQLDATE',
       'EventCode', 'SOURCEURL', 'NumMentions', 'NumSources', 'NumArticles',
       'valid_SOURCEURL', 'clean_text_list', 'NER_admin0_list',
       'NER_admin1_list', 'NER_admin2_list', 'compound_score_list',
       'neg_score_list', 'neu_score_list', 'pos_score_list',
       'fatalities_freq_list', 'displaced_freq_list', 'detained_freq_list',
       'injured_freq_list', 'sexual_violence_freq_list', 'torture_freq_lis
t',
       'economic_shocks_freq_list', 'agriculture_freq_list',
       'weather_freq_list', 'food_insecurity_freq_list',
       'fatalities_count_list', 'displaced_count_list', 'detained_count_lis
t',
       'injured_count_list', 'sexual_violence_count_list',
       'torture_count_list', 'sentiment.compound_list', 'sentiment.neg_lis
t',
       'sentiment.neu_list', 'sentiment.pos_list', 'pred_impact_type_list',
       'pred_urgency_list', 'pred_resource_food_list',
       'pred_resource_water_list', 'pred_resource_cash_aid_list',
       'pred_resource_healthcare_list', 'pred_resource_shelter_list',
       'pred_resource_livelihoods_list', 'pred_resource_education_list',
       'pred_resource_infrastructure_list', 'pred_resource_none_list'],
      dtype='object')
```

Out[2]:

	ADMIN0	ADMIN1	ADMIN2	CS_score	period	SQLDATE	EventCode	
271887	nigeria	zamfara	talata mafara	NaN	201907	[]	[20.0, 20.0, 43.0, 73.0, 46.0, 100.0, 43.0, 20...	[
271898	nigeria	zamfara	tsafe	NaN	201601	[]	[100.0, 42.0, 36.0, 100.0, 36.0, 20.0, 20.0, 1...	[http://
272100	niger	zinder	tanout	NaN	202308	[]	[42.0, 46.0, 42.0, 10.0, 10.0, 42.0, 172.0, 20...	[h

In [3]:
```
# Forecasting Configuration
# ============================
# Set the forecast horizon: predict CS_score N periods ahead
# Options: 1 (nowcasting), 2, 3, or 4 (forecasting)
# Literature shows GDELT features are more valuable for forecasting (2-4 per
# because baseline (previous_CS) becomes less powerful with longer horizons

FORECAST_HORIZON = 2  # Predict N periods ahead: 2 = ~8 months, 3 = ~1 year

print(f"=== Forecasting Configuration ===")
print(f"Forecast Horizon: {FORECAST_HORIZON} period(s) ahead")
if FORECAST_HORIZON == 1:
    print("Mode: NOWCASTING (predicting next period)")
else:
    print(f"Mode: FORECASTING (predicting {FORECAST_HORIZON} periods ahead)"
print(f"\nWhy forecasting helps GDELT features:")
print(f"  - Baseline 'previous_CS' becomes less predictive ({FORECAST_HORIZO
print(f"  - GDELT temporal patterns can capture early warning signals")
print(f"  - More realistic for early warning systems\n")
```

```
=== Forecasting Configuration ===
Forecast Horizon: 2 period(s) ahead
Mode: FORECASTING (predicting 2 periods ahead)

Why forecasting helps GDELT features:
  - Baseline 'previous_CS' becomes less predictive (2 periods old)
  - GDELT temporal patterns can capture early warning signals
  - More realistic for early warning systems
```

In [4]:
```
df = events.copy()
print(f"Total rows: {len(df.index)}")
print(f"Period range: {min(df['period'])} to {max(df['period'])}")
print(f"Columns: {len(df.columns)}")

# Drop rows where ADMIN0 is missing
mask_missing = df['ADMIN0'].isna() | (df['ADMIN0'] == 'None') | (df['ADMIN0'
```

```
if mask_missing.sum() > 0:
    print(f"Dropping {mask_missing.sum()} rows with ADMIN0=None")
    df = df[~mask_missing].copy()

print(f"Final rows: {len(df)}")
df.tail()
```

```
Total rows: 71293
Period range: 201601 to 202406
Columns: 51
Dropping 429 rows with ADMIN0=None
Final rows: 70864
```

Out[4]:

	ADMIN0	ADMIN1	ADMIN2	CS_score	period	SQLDATE	EventCode	
271876	nigeria	zamfara	talata mafara	NaN	201804	[]	[10.0, 42.0, 71.0, 43.0, 43.0, 20.0, 43.0, 43....	[I
271883	nigeria	zamfara	talata mafara	NaN	201901	[]	[42.0, 10.0, 10.0, 20.0, 10.0, 10.0, 10.0, 100...	
271887	nigeria	zamfara	talata mafara	NaN	201907	[]	[20.0, 20.0, 43.0, 73.0, 46.0, 100.0, 43.0, 20...	[
271898	nigeria	zamfara	tsafe	NaN	201601	[]	[100.0, 42.0, 36.0, 100.0, 36.0, 20.0, 20.0, 1...	[http://
272100	niger	zinder	tanout	NaN	202308	[]	[42.0, 46.0, 42.0, 10.0, 10.0, 42.0, 172.0, 20...	[h

In [5]:

```
key_cols = ["ADMIN0", "ADMIN1", "ADMIN2", "period"]

def check_duplicates(df, name):
    dupes = (
        df
        .groupby(key_cols)
        .size()
        .reset_index(name="n")
        .query("n > 1")
    )
    print(f"{name}: {len(dupes)} duplicated keys")
    return dupes

dupes = check_duplicates(df, "events")
```

```
events: 19 duplicated keys
```

In [6]:

```python
# BETWEEN-ASSESSMENT FORECASTING SETUP
# ====================================
# CRITICAL: This implements the correct "between-assessment forecasting" set
#
# Key principle: Observed IPC defines targets; unobserved months are predict
#
# Target: y_next = next observed IPC assessment (forward-lagged label)
# Feature: IPC_last = last observed IPC before this row (forward-filled, cor
#
# Rules:
# - Only use IPC_last as IPC feature (no rolling, no multiple lags, no trend
# - CS_score is NOT the direct target (y_next is)
# - Keep ALL rows (including last periods without future assessments)
# - Filter to rows with valid y_next only when training models

print("="*70)
print("BETWEEN-ASSESSMENT FORECASTING SETUP")
print("="*70)
print("\nKey principle: Observed IPC defines targets; unobserved months are

# Create region identifiers
df['region'] = df['ADMIN0'] + '-' + df['ADMIN1']
df['district'] = df['ADMIN0'] + '-' + df['ADMIN1'] + '-' + df['ADMIN2']

# Sort by district (ADMIN2) and period to ensure deterministic operations
df = df.sort_values(['ADMIN0', 'ADMIN1', 'ADMIN2', 'period']).reset_index(dr

# CS_score is already in the events dataset - just ensure numeric
df['CS_score'] = pd.to_numeric(df['CS_score'], errors='coerce')

# Create IPC_last: last observed IPC before this row (forward-filled)
# This is the ONLY IPC feature allowed - it's stale and constant between ass
print("1. Creating IPC_last (last observed IPC, forward-filled)...")
df['IPC_last'] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False)['CS_
    lambda x: x.ffill()
)

# Create y_next: next observed IPC assessment (forward-lagged label)
# This is the target variable - we predict the next assessment outcome
print("2. Creating y_next (next observed IPC assessment - THE TARGET)...")

def get_nth_assessment(group, horizon=1):
    """Get the Nth future observed IPC assessment for each row."""
    cs_values = group['CS_score'].values
    result = []

    for i in range(len(cs_values)):
        count = 0
        found = False
        for j in range(i + 1, len(cs_values)):
            if pd.notna(cs_values[j]) and 1 <= cs_values[j] <= 5:
                count += 1
                if count == horizon:
                    result.append(cs_values[j])
```

```
                    found = True
                    break
        if not found:
            result.append(np.nan)

    return pd.Series(result, index=group.index)

FORECAST_HORIZON = 2  # assessments ahead: 2 = ~8 months, 3 = ~1 year

df['y_next'] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False, group_
    get_nth_assessment, horizon=1
)
for h in [2, 3]:
    col = f'y_next_{h}'
    df[col] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False, group_k
        get_nth_assessment, horizon=h
    )

target_col = f'y_next_{FORECAST_HORIZON}'
print(f"\nForecast horizon: {FORECAST_HORIZON} assessments ahead (~{FORECAST
print(f"Rows with y_next:   {df['y_next'].notna().sum():,}")
print(f"Rows with y_next_2: {df['y_next_2'].notna().sum():,}")
print(f"Rows with y_next_3: {df['y_next_3'].notna().sum():,}")
print(f"Using target column: {target_col}")

# Keep ALL rows - don't drop periods without future assessments
# Rows without y_next (like 202402) can still be used for feature engineerin
# Filter to rows with valid y_next only when training models
rows_with_y_next = df['y_next'].notna().sum()
rows_without_y_next = df['y_next'].isna().sum()
df = df[df['IPC_last'].isin([1, 2, 3, 4, 5])]
print(f"   Rows with y_next (can be used for training): {rows_with_y_next}")
print(f"   Rows without y_next (last periods, feature engineering only): {ro
print(f"   Total rows kept: {len(df)} (no periods dropped)")

# Clean y_next where it exists: ensure valid range (1-5) and convert to int
# Only clean where y_next is not NaN
mask_valid_y_next = df['y_next'].notna()
df.loc[mask_valid_y_next, 'y_next'] = df.loc[mask_valid_y_next, 'y_next'].ro
df.loc[mask_valid_y_next, 'y_next'] = df.loc[mask_valid_y_next, 'y_next'].cl
df.loc[mask_valid_y_next, 'y_next'] = df.loc[mask_valid_y_next, 'y_next'].as

# Summary statistics
print(f"\n=== Summary ===")
print(f"IPC_last: last observed IPC (forward-filled, constant between assess
print(f"y_next: next observed IPC assessment (THE TARGET)")
print(f"\nIPC_last distribution:")
print(df['IPC_last'].value_counts().sort_index())
print(f"\ny_next distribution (TARGET):")
print(df['y_next'].value_counts().sort_index())
print(f"\nRows with valid y_next: {df['y_next'].notna().sum()}")
print(f"Rows without y_next (last periods): {df['y_next'].isna().sum()}")
print(f"Current DataFrame shape: {df.shape}")
print(f"Period range: {df['period'].min()} to {df['period'].max()}")

# Show example for verification
```

```
print(f"\n=== Example (first ADMIN2) ===")
if len(df) > 0:
    example_admin2 = df['ADMIN2'].iloc[0]
    example = df[df['ADMIN2'] == example_admin2].head(12)[['ADMIN2', 'perioc
    print(example.to_string(index=False))
    print(f"\nNote: IPC_last is constant between assessments, y_next is the

print("\n" + "="*70)
```

```
======================================================================
BETWEEN-ASSESSMENT FORECASTING SETUP
======================================================================

Key principle: Observed IPC defines targets; unobserved months are predictio
n times.

1. Creating IPC_last (last observed IPC, forward-filled)...
2. Creating y_next (next observed IPC assessment - THE TARGET)...
/var/folders/3y/cdj5wjs107z_64y5f9ws4cj40000gn/T/ipykernel_64497/1384409131.
py:64: FutureWarning: DataFrameGroupBy.apply operated on the grouping column
s. This behavior is deprecated, and in a future version of pandas the groupi
ng columns will be excluded from the operation. Either pass `include_groups=
False` to exclude the groupings or explicitly select the grouping columns af
ter groupby to silence this warning.
  df['y_next'] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False, grou
p_keys=False).apply(
/var/folders/3y/cdj5wjs107z_64y5f9ws4cj40000gn/T/ipykernel_64497/1384409131.
py:69: FutureWarning: DataFrameGroupBy.apply operated on the grouping column
s. This behavior is deprecated, and in a future version of pandas the groupi
ng columns will be excluded from the operation. Either pass `include_groups=
False` to exclude the groupings or explicitly select the grouping columns af
ter groupby to silence this warning.
  df[col] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False, group_key
s=False).apply(
```

```
Forecast horizon: 2 assessments ahead (~8 months)
Rows with y_next:   66,446
Rows with y_next_2: 62,999
Rows with y_next_3: 59,735
Using target column: y_next_2
   Rows with y_next (can be used for training): 66446
   Rows without y_next (last periods, feature engineering only): 4418
   Total rows kept: 69645 (no periods dropped)

=== Summary ===
IPC_last: last observed IPC (forward-filled, constant between assessments)
y_next: next observed IPC assessment (THE TARGET)

IPC_last distribution:
IPC_last
1.0    34059
2.0    18841
3.0    14936
4.0     1805
5.0        4
Name: count, dtype: int64

y_next distribution (TARGET):
y_next
1.0    30949
2.0    18339
3.0    14603
4.0     1754
5.0        4
Name: count, dtype: int64

Rows with valid y_next: 65649
Rows without y_next (last periods): 3996
Current DataFrame shape: (69645, 57)
Period range: 201602 to 202406

=== Example (first ADMIN2) ===
ADMIN2 period  CS_score  IPC_last  y_next
  bale 201602       1.0       1.0     1.0
  bale 201606       1.0       1.0     1.0
  bale 201610       1.0       1.0     1.0
  bale 201702       1.0       1.0     1.0
  bale 201706       1.0       1.0     1.0
  bale 201710       1.0       1.0     1.0
  bale 202012       NaN       1.0     1.0
  bale 202106       1.0       1.0     1.0
  bale 202110       1.0       1.0     1.0
  bale 202202       1.0       1.0     1.0
  bale 202306       1.0       1.0     1.0
  bale 202310       1.0       1.0     1.0

Note: IPC_last is constant between assessments, y_next is the next assessmen
t value

======================================================================
```

```
/var/folders/3y/cdj5wjs107z_64y5f9ws4cj40000gn/T/ipykernel_64497/1384409131.
py:69: FutureWarning: DataFrameGroupBy.apply operated on the grouping column
s. This behavior is deprecated, and in a future version of pandas the groupi
ng columns will be excluded from the operation. Either pass `include_groups=
False` to exclude the groupings or explicitly select the grouping columns af
ter groupby to silence this warning.
  df[col] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False, group_key
s=False).apply(
```

In [7]:

```
# Article coverage by IPC score and ADMIN0
# ========================================

df_ipc = df[df['IPC_last'].notna()].copy()
df_ipc['has_articles'] = df_ipc['valid_SOURCEURL'].apply(
    lambda x: x.size > 0 if isinstance(x, np.ndarray) else bool(x) if x is r
)

total = len(df_ipc)
print(f"Total rows (with IPC_last): {total:,}")
print(f"Rows with articles: {df_ipc['has_articles'].sum():,} ({df_ipc['has_a

# Breakdown by IPC_last
print("\nBreakdown by IPC_last:")
breakdown = df_ipc.groupby('IPC_last').agg(
    total=('has_articles', 'size'),
    with_articles=('has_articles', 'sum'),
)
breakdown['pct_articles'] = (breakdown['with_articles'] / breakdown['total']
print(breakdown)

# Breakdown by ADMIN0
print("\nBreakdown by ADMIN0:")
breakdown_admin0 = df_ipc.groupby('ADMIN0').agg(
    total=('has_articles', 'size'),
    with_articles=('has_articles', 'sum'),
    admin2_total=('ADMIN2', 'nunique'),
)
admin2_cov = df_ipc[df_ipc['has_articles']].groupby('ADMIN0')['ADMIN2'].nuni
breakdown_admin0 = breakdown_admin0.join(admin2_cov, how='left').fillna({'ad
breakdown_admin0['admin2_with_coverage'] = breakdown_admin0['admin2_with_cov
breakdown_admin0['pct_admin2_covered'] = (breakdown_admin0['admin2_with_cove
breakdown_admin0['pct_articles'] = (breakdown_admin0['with_articles'] / brea
breakdown_admin0 = breakdown_admin0.sort_values('total', ascending=False)
print(breakdown_admin0)

# Admin1 propagation gain
admin1_coverage = df_ipc.groupby(['ADMIN0', 'ADMIN1', 'period'])['has_articl
o, p = df_ipc['has_articles'].sum(), admin1_coverage.sum()
print(f"\nAdmin1 propagation: {o:,} ({o/total*100:.1f}%) -> {p:,} ({p/total*

# Breakdown by ADMIN0 after admin1 propagation
print("\nBreakdown by ADMIN0 (after admin1 propagation):")
df_ipc['has_articles_admin1'] = admin1_coverage.values
breakdown_prop = df_ipc.groupby('ADMIN0').agg(
    total=('has_articles', 'size'),
    admin2_total=('ADMIN2', 'nunique'),
```

```
    pct_admin2=('has_articles', 'mean'),
    pct_admin1=('has_articles_admin1', 'mean'),
)
breakdown_prop['pct_admin2'] = (breakdown_prop['pct_admin2'] * 100).round(1)
breakdown_prop['pct_admin1'] = (breakdown_prop['pct_admin1'] * 100).round(1)
breakdown_prop['gain_pp'] = (breakdown_prop['pct_admin1'] - breakdown_prop['
breakdown_prop = breakdown_prop.sort_values('total', ascending=False)
print(breakdown_prop)
```

Total rows (with IPC_last): 69,645
Rows with articles: 14,939 (21.5%)

Breakdown by IPC_last:

IPC_last	total	with_articles	pct_articles
1.0	34059	6964	20.4
2.0	18841	4243	22.5
3.0	14936	3274	21.9
4.0	1805	454	25.2
5.0	4	4	100.0

Breakdown by ADMIN0:

ADMIN0	total	with_articles	admin2_total	admin2_with_coverage	pct_admin2_covered
nigeria	19989	3133	729	364	49.9
kenya	7183	1467	269	171	63.6
yemen	6586	980	300	130	43.3
sudan	4522	942	167	51	30.5
mozambique	3861	502	154	58	37.7
uganda	3142	870	112	76	67.9
democratic republic of the congo	2772	242	181	41	22.7
ethiopia	2428	862	76	53	69.7
zimbabwe	2414	1211	74	64	86.5
somalia	2339	799	74	51	68.9
niger	1917	303	72	24	33.3
south sudan	1879	477	79	28	35.4
chad	1831	290	68	21	30.9
burundi	1498	92	111		
mali	1464	459	50		
malawi	854	430	27		
cameroon	838	176	58		
zambia	837	355	89		
burkina faso	794	260	45		
liberia	723	216	131		
madagascar	671	271	22		
mauritania	505	165	45		
guinea	333	212	34		
sierra leone	265	225	13		

```
burundi                                          22               19.8
mali                                             40               80.0
malawi                                           25               92.6
cameroon                                         36               62.1
zambia                                           53               59.6
burkina faso                                     28               62.2
liberia                                          18               13.7
madagascar                                       20               90.9
mauritania                                       12               26.7
guinea                                           12               35.3
sierra leone                                     10               76.9

                                 pct_articles
ADMIN0
nigeria                                  15.7
kenya                                    20.4
yemen                                    14.9
sudan                                    20.8
mozambique                               13.0
uganda                                   27.7
democratic republic of the congo          8.7
ethiopia                                 35.5
zimbabwe                                 50.2
somalia                                  34.2
niger                                    15.8
south sudan                              25.4
chad                                     15.8
burundi                                   6.1
mali                                     31.4
malawi                                   50.4
cameroon                                 21.0
zambia                                   42.4
burkina faso                             32.7
liberia                                  29.9
madagascar                               40.4
mauritania                               32.7
guinea                                   63.7
sierra leone                             84.9

Admin1 propagation: 14,939 (21.5%) -> 46,010 (66.1%), gain +31,071 (44.6%)

Breakdown by ADMIN0 (after admin1 propagation):
                                 total  admin2_total  pct_admin2  pct_admin
1  \
ADMIN0
nigeria                          19989           729        15.7        76.
8
kenya                             7183           269        20.4        54.
3
yemen                             6586           300        14.9        67.
7
sudan                             4522           167        20.8        58.
4
mozambique                        3861           154        13.0        54.
6
uganda                            3142           112        27.7        99.
```

democratic republic of the congo	2772	181	8.7	31.6
ethiopia	2428	76	35.5	82.5
zimbabwe	2414	74	50.2	94.2
somalia	2339	74	34.2	64.0
niger	1917	72	15.8	49.0
south sudan	1879	79	25.4	75.6
chad	1831	68	15.8	21.9
burundi	1498	111	6.1	23.6
mali	1464	50	31.4	68.2
malawi	854	27	50.4	98.2
cameroon	838	58	21.0	59.3
zambia	837	89	42.4	71.3
burkina faso	794	45	32.7	44.1
liberia	723	131	29.9	40.8
madagascar	671	22	40.4	64.4
mauritania	505	45	32.7	38.8
guinea	333	34	63.7	69.4
sierra leone	265	13	84.9	89.8

ADMIN0	gain_pp
nigeria	61.1
kenya	33.9
yemen	52.8
sudan	37.6
mozambique	41.6
uganda	71.3
democratic republic of the congo	22.9
ethiopia	47.0
zimbabwe	44.0
somalia	29.8
niger	33.2
south sudan	50.2
chad	6.1
burundi	17.5
mali	36.8
malawi	47.8

```
cameroon                    38.3
zambia                      28.9
burkina faso                11.4
liberia                     10.9
madagascar                  24.0
mauritania                   6.1
guinea                       5.7
sierra leone                 4.9
```

In [8]:
```
print(min(df['period']))
print(max(df['period']))
df = df.sort_values(['ADMIN0', 'ADMIN1', 'ADMIN2', 'period'])
df.tail()
```

```
201602
202406
```

Out[8]:

	ADMIN0	ADMIN1	ADMIN2	CS_score	period	SQLDATE	EventCode
70859	zimbabwe	midlands	zvishavane urban	2.0	202206	[]	[70.0, 70.0, 42.0]
70860	zimbabwe	midlands	zvishavane urban	2.0	202210	[]	[10.0, 173.0, 173.0, 10.0, 42.0]
70861	zimbabwe	midlands	zvishavane urban	2.0	202302	[]	[173.0, 112.0]
70862	zimbabwe	midlands	zvishavane urban	2.0	202306	[20230604.0]	[173.0, 71.0, 10.0, 30.0, 43.0, 71.0]
70863	zimbabwe	midlands	zvishavane urban	2.0	202402	[]	[173.0, 42.0, 10.0]

Feature engineering

In [9]:
```
# COMPREHENSIVE FEATURE ENGINEERING
# ==================================
# This cell performs ALL feature engineering for IPC and GDELT Events featur
# At this point: lists are NOT aggregated yet, y_next and IPC_last exist

print("="*70)
print("COMPREHENSIVE FEATURE ENGINEERING")
print("="*70)
print(f"\nStarting with {len(df)} rows")
print(f"Columns before feature engineering: {len(df.columns)}\n")
```

```python
# ============================================================================
# 1. AGGREGATE GDELT LIST FEATURES
# ============================================================================
print("1. Aggregating GDELT list features...")

# Helper functions (already defined in previous cell, but ensure they exist)
def safe_list_agg(lst, func):
    """Safely aggregate a list, handling None, empty lists, and non-numeric
    if lst is None:
        return np.nan
    if isinstance(lst, np.ndarray):
        if lst.size == 0:
            return np.nan
        lst = lst.tolist()
    try:
        if pd.isna(lst):
            return np.nan
    except (ValueError, TypeError):
        pass
    if isinstance(lst, (int, float)):
        return float(lst)
    if isinstance(lst, str):
        try:
            if lst.startswith('[') or lst.startswith('('):
                lst = eval(lst)
            else:
                return float(lst)
        except:
            return np.nan
    if not isinstance(lst, (list, tuple)):
        return np.nan
    if len(lst) == 0:
        return np.nan
    try:
        numeric_lst = []
        for x in lst:
            try:
                if pd.isna(x) or x is None:
                    continue
            except (ValueError, TypeError):
                pass
            try:
                val = float(x)
                if not np.isinf(val) and not np.isnan(val):
                    numeric_lst.append(val)
            except (ValueError, TypeError):
                continue
        if len(numeric_lst) == 0:
            return np.nan
        result = func(numeric_lst)
        return float(result) if not np.isnan(result) and not np.isinf(result
    except Exception as e:
        return np.nan

def safe_list_count(x):
```

```
    """Safely count elements in a list/array"""
    if x is None:
        return 0
    if isinstance(x, np.ndarray):
        return x.size if x.size > 0 else 0
    if isinstance(x, (list, tuple)):
        return len(x)
    try:
        if pd.isna(x):
            return 0
        return 1
    except (ValueError, TypeError):
        return 1

def aggregate_list_features(df, list_cols, prefix=""):
    """Aggregate list columns into meaningful statistical features for NLP-d

    For noisy NLP features, we keep only:
    - mean: central tendency
    - count: volume indicator
    - sum: total (especially useful for frequencies/counts)
    - max: peak intensity (only for crisis indicators)

    We skip: min (usually 0), std (variance not meaningful for noisy signals
    """
    new_cols = {}
    cols_to_drop = []

    for col in list_cols:
        if col not in df.columns:
            continue

        base_name = col.replace('_list', '')
        if prefix:
            base_name = f"{prefix}_{base_name}"

        # For text columns (NER, clean_text), only create count
        if 'NER' in col or 'clean_text' in col:
            new_cols[f"{base_name}_count"] = df[col].apply(safe_list_count)
        else:
            # For numeric columns, create meaningful aggregations only
            # Mean: central tendency (most important for noisy signals)
            new_cols[f"{base_name}_mean"] = df[col].apply(lambda x: safe_lis

            # Count: volume indicator (how many articles/events)
            new_cols[f"{base_name}_count"] = df[col].apply(safe_list_count)

            # Sum: total (especially useful for frequencies and counts)
            new_cols[f"{base_name}_sum"] = df[col].apply(lambda x: safe_list

            # Max: peak intensity (only for crisis indicators where peaks ma
            # Check if this is a crisis-related feature
            is_crisis_feature = any(x in col.lower() for x in ['fatalities',
                                                               'violence',
            if is_crisis_feature:
                new_cols[f"{base_name}_max"] = df[col].apply(lambda x: safe_
```

```
            # Skip: min (usually 0), std (variance not meaningful for noisy

        cols_to_drop.append(col)

    if new_cols:
        new_df = pd.DataFrame(new_cols, index=df.index)
        df = pd.concat([df, new_df], axis=1)

    if cols_to_drop:
        df = df.drop(columns=cols_to_drop)

    return df

# Find and aggregate list columns (events only, no merge suffix)
list_cols = [c for c in df.columns if '_list' in c]
print(f"   Found {len(list_cols)} list columns")

if list_cols:
    print("   Aggregating events features...")
    df = aggregate_list_features(df, list_cols, prefix="evt")

# ================================================================================
# 2. CLEAN AND PREPARE IPC FEATURES
# ================================================================================
print("\n2. Cleaning IPC features...")

# Clean IPC_last: round to nearest integer and ensure valid range
df['IPC_last'] = df['IPC_last'].round()
# Keep IPC_last as float for now (can filter invalid values later if needed)
# Don't drop rows here - keep all periods

print(f"   IPC_last range: {df['IPC_last'].min():.1f} to {df['IPC_last'].max
print(f"   Valid IPC_last (1-5): {(df['IPC_last'].between(1, 5, inclusive='b

# ================================================================================
# 3. CREATE TEMPORAL FEATURES FOR GDELT
# ================================================================================
print("\n3. Creating temporal features for GDELT...")

# Focus on most reliable features for temporal patterns
# Prioritize: sentiment, crisis indicators, food security
key_gdelt_features = [
    # Sentiment (most reliable NLP signals)
    'evt_compound_score_mean', 'evt_sentiment.compound_mean',

    # Crisis indicators (high signal-to-noise)
    'evt_fatalities_freq_mean', 'evt_displaced_freq_mean',

    # Food security (directly relevant to IPC)
    'evt_food_insecurity_freq_mean',

    # Economic/agricultural (relevant but less reliable)
    'evt_economic_shocks_freq_mean', 'evt_agriculture_freq_mean',
]
```

```
# Filter to features that actually exist
key_gdelt_features = [f for f in key_gdelt_features if f in df.columns]

print(f"   Creating temporal features for {len(key_gdelt_features)} key GDEL

# Ensure sorted for lag operations
df = df.sort_values(['ADMIN0', 'ADMIN1', 'ADMIN2', 'period']).reset_index(dr

# Create rolling windows (smoothing noise - most useful for NLP features)
# Use 3 and 6 periods to capture short and medium-term trends
for window in [3, 6]:
    for feat in key_gdelt_features:
        df[f"{feat}_rolling_{window}"] = df.groupby(['ADMIN0', 'ADMIN1', 'AD
            lambda x: x.rolling(window=window, min_periods=1).mean()
        )

# Create lags (1 and 2 periods only - reduce noise)
# Lags can capture trends but too many lags add noise
for lag in [1, 2]:
    for feat in key_gdelt_features:
        df[f"{feat}_lag{lag}"] = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'],

# Create escalation indicators (binary - less noisy than continuous change)
# Use rolling mean comparison instead of raw change to reduce noise
for feat in key_gdelt_features:
    rolling_3 = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False)[feat]
        lambda x: x.rolling(window=3, min_periods=1).mean()
    )
    rolling_6 = df.groupby(['ADMIN0', 'ADMIN1', 'ADMIN2'], sort=False)[feat]
        lambda x: x.rolling(window=6, min_periods=2).mean()
    )
    # Escalation: current 3-period average > 6-period average
    df[f"{feat}_escalation"] = (rolling_3 > rolling_6).astype(int)

# Skip: change, pct_change, anomaly (too noisy for NLP-derived features)

print(f"   Created temporal features: rolling windows (3, 6 periods), lags (

# ============================================================================
# 4. CREATE FEATURE INDICATORS AND INTERACTIONS
# ============================================================================
print("\n4. Creating feature indicators and interactions...")

# GDELT feature availability indicator
gdelt_cols = [c for c in df.columns if c.startswith('evt_')]
if gdelt_cols:
    has_gdelt = df[gdelt_cols].notna().any(axis=1)
    df['has_gdelt'] = has_gdelt.astype(int)
else:
    df['has_gdelt'] = 0

# Sentiment features from events

# Combine crisis indicators
crisis_features = [c for c in df.columns if any(x in c for x in ['fatalities
                   and c.endswith('_mean') and c.startswith('evt_')]
```

```python
if crisis_features:
    df['crisis_intensity'] = df[crisis_features].fillna(0).sum(axis=1)

# Food security combined
food_features = [c for c in df.columns if 'food_insecurity' in c and c.endsw
                 and c.startswith('evt_')]
if food_features:
    df['food_security_combined'] = df[food_features].fillna(0).mean(axis=1)

print(f"   Created interaction and combined features")

# ============================================================================
# 5. CREATE GEOGRAPHIC DUMMY VARIABLES
# ============================================================================
print("\n5. Creating geographic dummy variables...")

# Only create dummies for base geographic identifiers (ADMIN0, ADMIN1, ADMIN
# Note: 'region' and 'district' are redundant (derived from ADMIN0+ADMIN1 ar
geographic_cols = []
for col in ['ADMIN0', 'ADMIN1', 'ADMIN2']:
    if col in df.columns:
        # Create dummy variables (one-hot encoding)
        dummies = pd.get_dummies(df[col], prefix=col, drop_first=False)
        df = pd.concat([df, dummies], axis=1)
        geographic_cols.extend(dummies.columns.tolist())
        print(f"   Created {len(dummies.columns)} dummy variables for {col}"

geographic_features = geographic_cols
print(f"   Total geographic features: {len(geographic_features)}")
print(f"   Note: 'region' and 'district' are kept as categorical identifiers

# ============================================================================
# 6. SUMMARY
# ============================================================================
print("\n" + "="*70)
print("FEATURE ENGINEERING COMPLETE")
print("="*70)

print(f"\nFinal dataset: {len(df)} rows")
print(f"Total columns: {len(df.columns)}")

# Feature counts
ipc_features = ['IPC_last']

# GDELT base features: evt_ prefixed features
gdelt_base = [c for c in df.columns if c.startswith('evt_')
              and not any(x in c for x in ['_lag', '_rolling', '_escalation'

# GDELT temporal features: evt_ prefixed features with temporal patterns
gdelt_temporal = [c for c in df.columns if c.startswith('evt_')
                  and any(x in c for x in ['_lag', '_rolling', '_escalatior

# Geographic features were created as dummies in section 5 above
# geographic_features is already set to geographic_cols

# Geographic features: only ADMIN0, ADMIN1, ADMIN2 are converted to dummies
```

```
# 'region' and 'district' are kept as categorical identifiers for grouping b
# Exclude original categorical columns from feature lists (but keep them in
geographic_categorical_cols = ['ADMIN0', 'ADMIN1', 'ADMIN2', 'region', 'dist

# Exclude all non-feature columns (geographic categoricals, metadata, scores
excluded_cols = (ipc_features + gdelt_base + gdelt_temporal + geographic_fea
                 ['y_next', 'has_gdelt', 'has_articles',
                  'CS_score',
                  'period', 'SQLDATE', 'EventCode', 'SOURCEURL', 'NumMentior
                  'NumArticles', 'valid_SOURCEURL'] +
                 geographic_categorical_cols +
                 # Also exclude any remaining list columns that weren't aggr
                 [c for c in df.columns if '_list' in c])
other_features = [c for c in df.columns if c not in excluded_cols]

print(f"\nFeature breakdown:")
print(f"  - IPC features: {len(ipc_features)}")
print(f"  - GDELT base features: {len(gdelt_base)} (mean, count, sum, max fo
print(f"  - GDELT temporal features: {len(gdelt_temporal)} (rolling windows,
print(f"  - Geographic features: {len(geographic_features)} (dummy variables
print(f"  - Other features: {len(other_features)}")
print(f"  - Total features: {len(ipc_features) + len(gdelt_base) + len(gdelt
print(f"\nNote: Simplified feature set for noisy NLP-derived signals:")
print(f"  - Base: mean, count, sum (all features) + max (crisis features onl
print(f"  - Temporal: rolling windows (3, 6 periods), lags (1, 2 periods), e
print(f"  - Removed: min, std, change, pct_change, anomaly (too noisy for NL

print(f"\nRows with y_next: {df['y_next'].notna().sum()} ({df['y_next'].notr
print(f"Rows with IPC_last: {df['IPC_last'].notna().sum()} ({df['IPC_last'].
print(f"Rows with GDELT features: {df['has_gdelt'].sum()} ({df['has_gdelt'].

# ============================================================================
# 6. DROP PROCESSED/METADATA COLUMNS
# ============================================================================
print("\n6. Dropping processed metadata columns...")

cols_to_drop = ['SQLDATE', 'EventCode', 'SOURCEURL', 'NumMentions',
                'NumSources', 'NumArticles', 'valid_SOURCEURL']
# Only drop columns that exist
cols_to_drop = [c for c in cols_to_drop if c in df.columns]
if cols_to_drop:
    df.drop(cols_to_drop, axis=1, inplace=True)
    print(f"   Dropped {len(cols_to_drop)} metadata columns")

print("\n" + "="*70)
print("READY FOR MODELING")
print("="*70)
print("\nFeature sets available:")
print(f"  - IPC only: {len(ipc_features)} IPC features + {len(geographic_fea
print(f"  - IPC + GDELT base: {len(ipc_features)} IPC + {len(gdelt_base)} GD
print(f"  - IPC + GDELT + temporal: {len(ipc_features)} IPC + {len(gdelt_bas
print("="*70)
```

```
================================================================================
COMPREHENSIVE FEATURE ENGINEERING
================================================================================

Starting with 69645 rows
Columns before feature engineering: 57

1. Aggregating GDELT list features...
   Found 39 list columns
   Aggregating events features...

2. Cleaning IPC features...
   IPC_last range: 1.0 to 5.0
   Valid IPC_last (1-5): 69645 rows

3. Creating temporal features for GDELT...
   Creating temporal features for 7 key GDELT features
   Created temporal features: rolling windows (3, 6 periods), lags (1, 2 per
iods), escalation indicators

4. Creating feature indicators and interactions...
   Created interaction and combined features

5. Creating geographic dummy variables...
   Created 24 dummy variables for ADMIN0
   Created 338 dummy variables for ADMIN1
   Created 2969 dummy variables for ADMIN2
   Total geographic features: 3331
   Note: 'region' and 'district' are kept as categorical identifiers but not
converted to dummies (redundant with ADMIN0+ADMIN1+ADMIN2)

================================================================================
FEATURE ENGINEERING COMPLETE
================================================================================

Final dataset: 69645 rows
Total columns: 3506

Feature breakdown:
  - IPC features: 1
  - GDELT base features: 119 (mean, count, sum, max for crisis)
  - GDELT temporal features: 35 (rolling windows, lags, escalation)
  - Geographic features: 3331 (dummy variables)
  - Other features: 4
  - Total features: 3490

Note: Simplified feature set for noisy NLP-derived signals:
  - Base: mean, count, sum (all features) + max (crisis features only)
  - Temporal: rolling windows (3, 6 periods), lags (1, 2 periods), escalatio
n indicators
  - Removed: min, std, change, pct_change, anomaly (too noisy for NLP featur
es)

Rows with y_next: 65649 (94.3%)
Rows with IPC_last: 69645 (100.0%)
Rows with GDELT features: 69645 (100.0%)
```

```
6. Dropping processed metadata columns...
   Dropped 7 metadata columns

======================================================================
READY FOR MODELING
======================================================================

Feature sets available:
  - IPC only: 1 IPC features + 3331 geographic features
  - IPC + GDELT base: 1 IPC + 119 GDELT base + 3331 geographic
  - IPC + GDELT + temporal: 1 IPC + 119 GDELT base + 35 GDELT temporal + 333
1 geographic
======================================================================
```

In [10]:
```
print(min(df['period']))
print(max(df['period']))
df = df.sort_values(['ADMIN0', 'ADMIN1', 'ADMIN2', 'period'])
df.tail()
```

```
201602
202406
```

Out[10]:

	ADMIN0	ADMIN1	ADMIN2	CS_score	period	region	district	IPC
69640	zimbabwe	midlands	zvishavane urban	2.0	202206	zimbabwe-midlands	zimbabwe-midlands-zvishavane urban	
69641	zimbabwe	midlands	zvishavane urban	2.0	202210	zimbabwe-midlands	zimbabwe-midlands-zvishavane urban	
69642	zimbabwe	midlands	zvishavane urban	2.0	202302	zimbabwe-midlands	zimbabwe-midlands-zvishavane urban	
69643	zimbabwe	midlands	zvishavane urban	2.0	202306	zimbabwe-midlands	zimbabwe-midlands-zvishavane urban	
69644	zimbabwe	midlands	zvishavane urban	2.0	202402	zimbabwe-midlands	zimbabwe-midlands-zvishavane urban	

Configure train and test, and loops

In [11]:
```
# ======================================================================
# NAIVE BASELINE MODELS — 3-STEP-AHEAD (~1 YEAR) FORECAST
# ======================================================================
```

```python
# Prediction: at assessment T, predict CS_score at T+3 (3 assessments ahead)
#
# PPS     — No-Change: predict CS_score(T+3) = CS_score(T)
# SPLY    — Same Period Last Year: CS_score from ~12 months before T+3
# Max-2PP — Max of 2 Previous Periods: max CS_score at T and T-1
# ============================================================================

from sklearn.metrics import accuracy_score, precision_score, recall_score, f

admin_cols = ['ADMIN0', 'ADMIN1', 'ADMIN2']
H = FORECAST_HORIZON  # 3

assessment_periods_bl = sorted(
    df.loc[df['CS_score'].between(1, 5), 'period'].unique()
)

cs_lookup = (df.loc[df['CS_score'].between(1, 5), admin_cols + ['period', 'C
             .drop_duplicates(admin_cols + ['period']))

# Build evaluation pairs: (prediction_period, target_period)
eval_pairs = []
for i, pred_period in enumerate(assessment_periods_bl):
    target_idx = i + H
    if target_idx < len(assessment_periods_bl):
        eval_pairs.append((pred_period, assessment_periods_bl[target_idx]))

print(f"Forecast horizon: {H} assessments ahead")
print(f"Evaluation pairs: {len(eval_pairs)}  (predict from → target)")
for pred_p, tgt_p in eval_pairs[:5]:
    print(f"  {pred_p} → {tgt_p}")
if len(eval_pairs) > 5:
    print(f"  ... ({len(eval_pairs) - 5} more)")
print()

baseline_names = ['PPS', 'SPLY', 'Max-2PP']
period_results = {name: [] for name in baseline_names}

for pred_period, target_period in eval_pairs:
    # Features/predictions are based on data at pred_period
    pred_scores = cs_lookup[cs_lookup['period'] == pred_period].copy()

    # Target is CS_score at target_period
    target_scores = (cs_lookup[cs_lookup['period'] == target_period]
                     [admin_cols + ['CS_score']]
                     .rename(columns={'CS_score': 'target'}))

    # Only evaluate areas present in both periods
    merged = pred_scores.merge(target_scores, on=admin_cols, how='inner')
    if len(merged) == 0:
        continue

    y_test = merged['target'].astype(int)
    prior_periods = [p for p in assessment_periods_bl if p < pred_period]
    if len(prior_periods) < 1:
        continue
```

```
    # PPS: predict target = current CS_score (no change over 1 year)
    pps_pred = merged['CS_score'].fillna(0).astype(int)

    # SPLY: CS_score from ~12 months before the TARGET period
    target_sply = str(int(target_period) - 100)
    all_prior_to_target = [p for p in assessment_periods_bl if p <= pred_per
    sply_period = min(all_prior_to_target, key=lambda p: abs(int(p) - int(ta
    sply_lookup_df = (cs_lookup[cs_lookup['period'] == sply_period]
                      [admin_cols + ['CS_score']].rename(columns={'CS_score'
    sply_merged = merged.merge(sply_lookup_df, on=admin_cols, how='left')
    sply_pred = sply_merged['sply_pred'].fillna(0).astype(int)

    # Max-2PP: max CS_score from pred_period and the one before
    last2 = [pred_period] + prior_periods[-1:]
    max2pp_df = (cs_lookup[cs_lookup['period'].isin(last2)]
                 .groupby(admin_cols)['CS_score'].max().reset_index()
                 .rename(columns={'CS_score': 'max2pp_pred'}))
    max2pp_merged = merged.merge(max2pp_df, on=admin_cols, how='left')
    max2pp_pred = max2pp_merged['max2pp_pred'].fillna(0).astype(int)

    preds = {'PPS': pps_pred, 'SPLY': sply_pred, 'Max-2PP': max2pp_pred}

    for name in baseline_names:
        period_results[name].append({
            'pred_period': pred_period,
            'target_period': target_period,
            'n_test': len(merged),
            'accuracy': accuracy_score(y_test, preds[name]),
            'precision': precision_score(y_test, preds[name], average='weigh
            'recall': recall_score(y_test, preds[name], average='weighted',
            'f1': f1_score(y_test, preds[name], average='weighted', zero_div
        })

# ── Summary ──────────────────────────────────────────────────────────
rows = []
for name in baseline_names:
    res = period_results[name]
    if not res:
        continue
    rows.append({
        'Model': name,
        'Accuracy': np.mean([r['accuracy'] for r in res]),
        'Precision': np.mean([r['precision'] for r in res]),
        'Recall': np.mean([r['recall'] for r in res]),
        'F1': np.mean([r['f1'] for r in res]),
        'N Periods': len(res),
        'Total Test': sum(r['n_test'] for r in res),
    })

baseline_summary = pd.DataFrame(rows)
print(baseline_summary.to_string(index=False))
```

```
Forecast horizon: 2 assessments ahead
Evaluation pairs: 24  (predict from → target)
  201602 → 201610
  201606 → 201702
  201610 → 201706
  201702 → 201710
  201706 → 201802
  ... (19 more)

  Model  Accuracy  Precision   Recall       F1  N Periods  Total Test
    PPS  0.727398   0.740394 0.727398 0.724732         23       48194
   SPLY  0.686285   0.743649 0.686285 0.700476         23       48194
Max-2PP  0.734806   0.760531 0.734806 0.737406         23       48194
```

ML from here

In [ ]:
```python
n_training_periods = 20     # months of training data before each test asses
n_eval_periods = 26          # number of IPC assessment periods to evaluate

# Optional: set to None to auto-detect latest assessment, or specify e.g. 20
start_period = None
```

In [13]:
```python
# ================================================================================
# ROLLING WINDOW EVALUATION — 3-STEP-AHEAD IPC FORECAST (~1 YEAR)
# ================================================================================
# At assessment period T, predict CS_score at T+3 (3 assessments ahead).
# Train on y_next_3 (3rd future assessment's score).
# No IPC_last leakage: IPC_last = current IPC, target = IPC 1 year ahead.
# ================================================================================

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.base import clone
from sklearn.metrics import accuracy_score, precision_score, recall_score, f
from catboost import CatBoostClassifier

all_periods = sorted(df['period'].unique())

# ── 1. Identify IPC assessment periods ──────────────────────────────────────
ipc_obs_per_period = df[df['CS_score'].between(1, 5)].groupby('period').size
assessment_periods = sorted(ipc_obs_per_period.index.tolist())

H = FORECAST_HORIZON  # 3

print("="*70)
print(f"IPC ASSESSMENT PERIODS — {H}-STEP-AHEAD FORECAST")
print("="*70)
print(f"\nTotal periods in data: {len(all_periods)}  ({all_periods[0]} → {al
print(f"Assessment periods (with CS_score data): {len(assessment_periods)}\r
for p in assessment_periods:
    print(f"  {p}:  {ipc_obs_per_period[p]:>5,} scored")

# ── 2. Build prediction→target pairs (T → T+H) ──────────────────────────────
```

```
all_eval_pairs = []
for i, pred_period in enumerate(assessment_periods):
    target_idx = i + H
    if target_idx < len(assessment_periods):
        all_eval_pairs.append((pred_period, assessment_periods[target_idx]))

if start_period is not None:
    all_eval_pairs = [(p, t) for p, t in all_eval_pairs if p <= start_period

eval_pairs = all_eval_pairs[-n_eval_periods:]

print(f"\n{'='*70}")
print("ROLLING WINDOW EVALUATION SETUP")
print(f"{'='*70}")
print(f"\nConfiguration:")
print(f"  - Forecast horizon:      {H} assessments ahead (~{H * 4} months)")
print(f"  - Training window:       {n_training_periods} periods before predi
print(f"  - Evaluation pairs:      {len(eval_pairs)} (of {len(all_eval_pairs

# ── 3. Create evaluation splits ──────────────────────────────────────
evaluation_splits = []

for i, (pred_period, target_period) in enumerate(reversed(eval_pairs)):
    pred_idx = all_periods.index(pred_period)
    train_end_idx = pred_idx - 1
    train_start_idx = max(0, train_end_idx - n_training_periods + 1)

    if train_end_idx < 0:
        print(f"\n  ⚠️  Pair {pred_period}→{target_period}: no training data
        continue

    train_periods = all_periods[train_start_idx:train_end_idx + 1]

    evaluation_splits.append({
        'eval_id': i + 1,
        'pred_period': pred_period,
        'target_period': target_period,
        'n_scored_pred': int(ipc_obs_per_period.get(pred_period, 0)),
        'n_scored_target': int(ipc_obs_per_period.get(target_period, 0)),
        'train_periods': train_periods,
        'train_start': train_periods[0],
        'train_end': train_periods[-1],
        'n_train_periods': len(train_periods),
    })

print(f"\nEvaluation schedule (latest → earliest):\n")
for s in evaluation_splits:
    print(f"  Eval {s['eval_id']}: predict at {s['pred_period']} → target {s
          f"({s['n_scored_pred']:,} areas)  |  "
          f"train {s['train_start']} → {s['train_end']}  "
          f"({s['n_train_periods']} periods)")

print(f"\n{'='*70}")
print(f"Created {len(evaluation_splits)} evaluation splits")
print("="*70)
```

```
# ============================================================================
# DEFINE FEATURE SETS
# ============================================================================
if 'ipc_features' in globals():
    ipc_feat = [c for c in ipc_features if c in df.columns]
else:
    ipc_feat = ['IPC_last']

_temporal_suffixes = ('_rolling_', '_lag', '_escalation')

if 'gdelt_features' in globals():
    gdelt_base_feat = [c for c in gdelt_features if c in df.columns]
else:
    gdelt_base_feat = [c for c in df.columns
                       if c.startswith('evt_') and not any(s in c for s in _

if 'temporal_features' in globals():
    gdelt_temporal_feat = [c for c in temporal_features if c in df.columns]
else:
    gdelt_temporal_feat = [c for c in df.columns
                           if c.startswith('evt_') and any(s in c for s in _

if 'geographic_features' in globals():
    geographic_feat = [c for c in geographic_features if c in df.columns]
else:
    geographic_feat = [c for c in df.columns if c.startswith('geo_')]

feature_sets = {
    # With geographic dummies (location fixed effects)
    'IPC only':               ipc_feat + geographic_feat,
    'IPC + GDELT base':       ipc_feat + gdelt_base_feat + geographic_feat,
    'IPC + GDELT + temporal': ipc_feat + gdelt_base_feat + gdelt_temporal_fe
    'GDELT only':             gdelt_base_feat + gdelt_temporal_feat + geogra
    # Without geographic dummies (pure feature signal)
    'IPC only (no geo)':               ipc_feat,
    'IPC + GDELT base (no geo)':       ipc_feat + gdelt_base_feat,
    'IPC + GDELT + temporal (no geo)': ipc_feat + gdelt_base_feat + gdelt_te
    'GDELT only (no geo)':             gdelt_base_feat + gdelt_temporal_feat
}

print("\nFeature sets available:")
for name, features in feature_sets.items():
    n_ipc = len([c for c in features if c in ipc_feat])
    n_gdelt_base = len([c for c in features if c in gdelt_base_feat])
    n_gdelt_temp = len([c for c in features if c in gdelt_temporal_feat])
    n_geo = len([c for c in features if c in geographic_feat])
    print(f"  - {name}: {n_ipc} IPC + {n_gdelt_base} GDELT base + {n_gdelt_t

# ============================================================================
# DEFINE MODELS
# ============================================================================
models = {
    'Logistic Regression': {
        'model': LogisticRegression(max_iter=1000, random_state=42, solver='
        'needs_scaling': True,
    },
```

```
    'Random Forest': {
        'model': RandomForestClassifier(
            n_estimators=150, max_depth=12, min_samples_split=15,
            min_samples_leaf=8, max_features='sqrt',
            class_weight='balanced', random_state=42, n_jobs=-1
        ),
        'needs_scaling': False,
    },
    'CatBoost': {
        'model': CatBoostClassifier(
            iterations=300, depth=4, learning_rate=0.05,
            loss_function='MultiClass', auto_class_weights='Balanced',
            l2_leaf_reg=5, colsample_bylevel=0.7,
            random_seed=42, verbose=0
        ),
        'needs_scaling': False,
    },
}

print(f"\nModels to evaluate: {', '.join(models.keys())}")

# ============================================================================
# TRAIN AND EVALUATE
# ============================================================================
print("\n" + "="*70)
print("TRAINING AND EVALUATION")
print("="*70)

all_results = {}  # keyed by (model_name, feature_set_name)
rf_importances = {}  # keyed by feature_set_name → list of (feature_cols, im

for model_name, model_cfg in models.items():
    for feature_set_name, feature_cols in feature_sets.items():
        result_key = (model_name, feature_set_name)
        print(f"\n{'='*70}")
        print(f"MODEL: {model_name}  |  FEATURE SET: {feature_set_name}")
        print(f"{'='*70}")
        print(f"Using {len(feature_cols)} features")

        feature_results = []

        for split in evaluation_splits:
            print(f"\n  ── Eval {split['eval_id']}: predict at {split['pred_

            # ── Training set: all prior periods with valid target ────────
            train_df = df[
                (df['period'].isin(split['train_periods'])) &
                (df[target_col].notna())
            ].copy()

            # ── Test set: features from pred_period, target from target_per
            pred_df = df[
                (df['period'] == split['pred_period']) &
                (df['CS_score'].between(1, 5))
            ].copy()
            target_scores = (
```

```
            df.loc[(df['period'] == split['target_period']) & df['CS_scc
                   ['ADMIN0', 'ADMIN1', 'ADMIN2', 'CS_score']]
            .drop_duplicates(['ADMIN0', 'ADMIN1', 'ADMIN2'])
            .rename(columns={'CS_score': '_target_CS'})
        )
        test_df = pred_df.merge(target_scores, on=['ADMIN0', 'ADMIN1', '

        print(f"  Training samples:   {len(train_df):,}")
        print(f"  Test samples:       {len(test_df):,}  (areas in both {

        if len(train_df) == 0:
            print(f"  ⚠️  Skipping — no training data")
            continue
        if len(test_df) == 0:
            print(f"  ⚠️  Skipping — no test data")
            continue

        # ── Prepare features and target ──────────────────────────────
        # No IPC_last leakage: IPC_last = current IPC at pred_period,
        # target = CS_score at target_period (3 assessments ahead)
        X_train = train_df[feature_cols].fillna(0)
        y_train = train_df[target_col].astype(int)

        X_test = test_df[feature_cols].fillna(0)
        y_test = test_df['_target_CS'].astype(int)

        # ── Scale (only for models that need it) ─────────────────────
        if model_cfg['needs_scaling']:
            scaler = StandardScaler()
            X_train_fit = scaler.fit_transform(X_train)
            X_test_fit = scaler.transform(X_test)
        else:
            X_train_fit = X_train
            X_test_fit = X_test

        # ── Train ────────────────────────────────────────────────────
        print(f"  Training {model_name}...")
        model = clone(model_cfg['model'])
        model.fit(X_train_fit, y_train)

        y_pred = model.predict(X_test_fit)

        # ── Capture Random Forest feature importances ────────────────
        if model_name == 'Random Forest':
            if feature_set_name not in rf_importances:
                rf_importances[feature_set_name] = []
            rf_importances[feature_set_name].append(
                pd.Series(model.feature_importances_, index=feature_cols
            )

        # ── Metrics ──────────────────────────────────────────────────
        accuracy  = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='weighted',
        recall    = recall_score(y_test, y_pred, average='weighted', zer
        f1        = f1_score(y_test, y_pred, average='weighted', zero_di
        cm        = confusion_matrix(y_test, y_pred)
```

```
            feature_results.append({
                'eval_id': split['eval_id'],
                'pred_period': split['pred_period'],
                'target_period': split['target_period'],
                'n_train': len(train_df),
                'n_test': len(test_df),
                'accuracy': accuracy,
                'precision': precision,
                'recall': recall,
                'f1': f1,
                'confusion_matrix': cm
            })

            print(f"\n  Results  (n_test={len(test_df):,}):")
            print(f"    Accuracy:  {accuracy:.4f}")
            print(f"    Precision: {precision:.4f}")
            print(f"    Recall:    {recall:.4f}")
            print(f"    F1 Score:  {f1:.4f}")
            print(f"\n  Confusion Matrix:\n    {cm}")

        all_results[result_key] = feature_results

# ============================================================================
# SUMMARY
# ============================================================================
if any(len(v) > 0 for v in all_results.values()):
    print("\n" + "="*70)
    print("SUMMARY: AVERAGE RESULTS BY MODEL & FEATURE SET")
    print("="*70)

    summary_data = []
    for (model_name, feature_set_name), results in all_results.items():
        if len(results) == 0:
            continue

        avg_acc  = np.mean([r['accuracy']  for r in results])
        avg_prec = np.mean([r['precision'] for r in results])
        avg_rec  = np.mean([r['recall']    for r in results])
        avg_f1   = np.mean([r['f1']        for r in results])
        total_test = sum(r['n_test'] for r in results)

        print(f"\n{model_name} — {feature_set_name}:")
        print("-" * 70)
        print(f"  Average Metrics (across {len(results)} assessments, {total
        print(f"    Accuracy:  {avg_acc:.4f}")
        print(f"    Precision: {avg_prec:.4f}")
        print(f"    Recall:    {avg_rec:.4f}")
        print(f"    F1 Score:  {avg_f1:.4f}")

        print(f"\n  Per evaluation:")
        for r in results:
            print(f"    {r['pred_period']}→{r['target_period']}:  n={r['n_te

        n_features = len(feature_sets[feature_set_name])
        n_ipc = len([c for c in feature_sets[feature_set_name] if c in ipc_f
```

```
        n_gdelt_base = len([c for c in feature_sets[feature_set_name] if c i
        n_gdelt_temp = len([c for c in feature_sets[feature_set_name] if c i
        n_geo = len([c for c in feature_sets[feature_set_name] if c in geogr

        summary_data.append({
            'Model': model_name,
            'Feature Set': feature_set_name,
            'N Features': n_features,
            'IPC': n_ipc,
            'GDELT Base': n_gdelt_base,
            'GDELT Temporal': n_gdelt_temp,
            'Geographic': n_geo,
            'Accuracy': avg_acc,
            'Precision': avg_prec,
            'Recall': avg_rec,
            'F1 Score': avg_f1,
            'Total Test': total_test,
            'N Evals': len(results),
        })

    summary_df = pd.DataFrame(summary_data)
    summary_df = summary_df.sort_values('F1 Score', ascending=False)

    print(f"\n{'='*70}")
    print("SUMMARY TABLE")
    print("="*70)
    print(summary_df.to_string(index=False))

else:
    print("\n⚠️  No evaluations completed — check data availability")

# ============================================================================
# RANDOM FOREST — AVERAGE FEATURE IMPORTANCES
# ============================================================================
if rf_importances:
    print("\n" + "="*70)
    print("RANDOM FOREST: AVERAGE FEATURE IMPORTANCES")
    print("="*70)

    for fs_name, imp_list in rf_importances.items():
        avg_imp = pd.concat(imp_list, axis=1).mean(axis=1).sort_values(ascer
        print(f"\n── {fs_name} (averaged over {len(imp_list)} splits) ──")
        print(f"{'Feature':<55} {'Importance':>10}")
        print("-" * 67)
        for feat, imp in avg_imp.head(25).items():
            print(f"  {feat:<53} {imp:>10.4f}")
        if len(avg_imp) > 25:
            print(f"  ... ({len(avg_imp) - 25} more features omitted)")
```

```
======================================================================
IPC ASSESSMENT PERIODS — 2-STEP-AHEAD FORECAST
======================================================================

Total periods in data: 101  (201602 → 202406)
Assessment periods (with CS_score data): 26

  201602:  2,639 scored
  201606:  2,635 scored
  201610:  2,748 scored
  201702:  2,449 scored
  201706:  2,573 scored
  201710:  2,577 scored
  201802:  2,441 scored
  201806:  2,440 scored
  201810:  1,081 scored
  201902:  2,400 scored
  201906:  2,400 scored
  201910:  2,400 scored
  202002:  1,588 scored
  202006:  2,570 scored
  202010:  2,491 scored
  202102:  2,489 scored
  202106:  2,536 scored
  202110:  2,535 scored
  202202:  2,535 scored
  202206:  2,568 scored
  202210:  2,399 scored
  202302:  1,376 scored
  202306:  2,613 scored
  202310:  1,919 scored
  202402:  2,376 scored
  202406:  1,919 scored

======================================================================
ROLLING WINDOW EVALUATION SETUP
======================================================================

Configuration:
  - Forecast horizon:      2 assessments ahead (~8 months)
  - Training window:       20 periods before prediction assessment
  - Evaluation pairs:      7 (of 24 possible)

Evaluation schedule (latest → earliest):

  Eval 1: predict at 202310 → target 202406  (1,919 areas)  |  train 202202
→ 202309  (20 periods)
  Eval 2: predict at 202306 → target 202402  (2,613 areas)  |  train 202110
→ 202305  (20 periods)
  Eval 3: predict at 202302 → target 202310  (1,376 areas)  |  train 202106
→ 202301  (20 periods)
  Eval 4: predict at 202210 → target 202306  (2,399 areas)  |  train 202102
→ 202209  (20 periods)
  Eval 5: predict at 202206 → target 202302  (2,568 areas)  |  train 202010
→ 202205  (20 periods)
  Eval 6: predict at 202202 → target 202210  (2,535 areas)  |  train 202006
```

```
→ 202201  (20 periods)
  Eval 7: predict at 202110 → target 202206  (2,535 areas)  |  train 202002
→ 202109  (20 periods)

======================================================================
Created 7 evaluation splits
======================================================================

Feature sets available:
  - IPC only: 1 IPC + 0 GDELT base + 0 GDELT temporal + 3331 geographic = 33
32 total
  - IPC + GDELT base: 1 IPC + 119 GDELT base + 0 GDELT temporal + 3331 geogr
aphic = 3451 total
  - IPC + GDELT + temporal: 1 IPC + 119 GDELT base + 35 GDELT temporal + 333
1 geographic = 3486 total
  - GDELT only: 0 IPC + 119 GDELT base + 35 GDELT temporal + 3331 geographic
= 3485 total
  - IPC only (no geo): 1 IPC + 0 GDELT base + 0 GDELT temporal + 0 geographi
c = 1 total
  - IPC + GDELT base (no geo): 1 IPC + 119 GDELT base + 0 GDELT temporal + 0
geographic = 120 total
  - IPC + GDELT + temporal (no geo): 1 IPC + 119 GDELT base + 35 GDELT tempo
ral + 0 geographic = 155 total
  - GDELT only (no geo): 0 IPC + 119 GDELT base + 35 GDELT temporal + 0 geog
raphic = 154 total

Models to evaluate: Logistic Regression, Random Forest, CatBoost

======================================================================
TRAINING AND EVALUATION
======================================================================

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC only
======================================================================
Using 3332 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.6128
    Precision: 0.6334
    Recall:    0.6128
    F1 Score:  0.6053

  Confusion Matrix:
    [[414  66   0   0]
 [341 412  77   0]
 [  7 155 291  15]
 [  0   3  79  59]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
```

```
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Logistic Regression...

 Results  (n_test=2,313):
   Accuracy:  0.7454
   Precision: 0.7395
   Recall:    0.7454
   F1 Score:  0.7411

 Confusion Matrix:
   [[777 144   3   0]
[132 581  91   4]
[  6  89 343  34]
[  0  20  66  23]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Logistic Regression...

 Results  (n_test=982):
   Accuracy:  0.6283
   Precision: 0.7434
   Recall:    0.6283
   F1 Score:  0.6456

 Confusion Matrix:
   [[335 267   1   0]
[ 28 208  35   1]
[  6  23  72   2]
[  0   0   2   2]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7582
   Precision: 0.7868
   Recall:    0.7582
   F1 Score:  0.7563

 Confusion Matrix:
   [[566  50   1   0]
[355 612  65   2]
[  9  81 574   7]
[  0   0  10  67]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7464
```

```
    Precision: 0.7519
    Recall:    0.7464
    F1 Score:  0.7469

  Confusion Matrix:
    [[448 132   0   0]
 [114 467  29   0]
 [ 13  56 112   5]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Logistic Regression...

  Results  (n_test=2,321):
    Accuracy:  0.7243
    Precision: 0.7251
    Recall:    0.7243
    F1 Score:  0.7214

  Confusion Matrix:
    [[713 128   0   0]
 [245 528  45   1]
 [ 12 148 426  21]
 [  0   0  40  14]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Logistic Regression...

  Results  (n_test=2,490):
    Accuracy:  0.7402
    Precision: 0.7475
    Recall:    0.7402
    F1 Score:  0.7291

  Confusion Matrix:
    [[743  60   3   0]
 [231 634  52   0]
 [  7 216 454   1]
 [  1  18  58  12]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC + GDELT base
======================================================================
Using 3451 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.6102
```

```
  Precision: 0.6292
  Recall:    0.6102
  F1 Score:  0.6025

 Confusion Matrix:
   [[414  66   0   0]
[343 410  77   0]
[  7 153 288  20]
[  0   3  79  59]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Logistic Regression...

 Results  (n_test=2,313):
  Accuracy:  0.7345
  Precision: 0.7280
  Recall:    0.7345
  F1 Score:  0.7303

 Confusion Matrix:
   [[775 146   3   0]
[145 572  88   3]
[  4  99 330  39]
[  0  19  68  22]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Logistic Regression...

 Results  (n_test=982):
  Accuracy:  0.6232
  Precision: 0.7426
  Recall:    0.6232
  F1 Score:  0.6408

 Confusion Matrix:
   [[331 271   1   0]
[ 27 209  35   1]
[  6  25  70   2]
[  0   0   2   2]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
  Accuracy:  0.7586
  Precision: 0.7869
  Recall:    0.7586
  F1 Score:  0.7572

 Confusion Matrix:
```

```
   [[565  51   1   0]
[355 618  60   1]
[  8  85 571   7]
[  0   0  11  66]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7500
   Precision: 0.7558
   Recall:    0.7500
   F1 Score:  0.7510

 Confusion Matrix:
   [[444 136   0   0]
[110 471  29   0]
[ 12  52 117   5]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Logistic Regression...

 Results  (n_test=2,321):
   Accuracy:  0.7230
   Precision: 0.7230
   Recall:    0.7230
   F1 Score:  0.7199

 Confusion Matrix:
   [[713 128   0   0]
[248 520  49   2]
[ 12 144 431  20]
[  0   0  40  14]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training Logistic Regression...

 Results  (n_test=2,490):
   Accuracy:  0.7373
   Precision: 0.7391
   Recall:    0.7373
   F1 Score:  0.7267

 Confusion Matrix:
   [[745  57   2   2]
[237 627  52   1]
[  9 216 451   2]
[  1  19  56  13]]
```

```
======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC + GDELT + temporal
======================================================================
Using 3486 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.6149
    Precision: 0.6328
    Recall:    0.6149
    F1 Score:  0.6081

  Confusion Matrix:
    [[413  67   0   0]
 [338 418  73   1]
 [  6 155 286  21]
 [  0   3  75  63]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Logistic Regression...

  Results  (n_test=2,313):
    Accuracy:  0.7406
    Precision: 0.7352
    Recall:    0.7406
    F1 Score:  0.7365

  Confusion Matrix:
    [[776 145   3   0]
 [144 568  93   3]
 [  4  93 343  32]
 [  0  19  64  26]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training Logistic Regression...

  Results  (n_test=982):
    Accuracy:  0.6130
    Precision: 0.7426
    Recall:    0.6130
    F1 Score:  0.6307

  Confusion Matrix:
    [[320 282   1   0]
 [ 24 210  36   2]
 [  6  25  70   2]
 [  0   0   2   2]]
```

```
 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7566
   Precision: 0.7836
   Recall:    0.7566
   F1 Score:  0.7548

 Confusion Matrix:
   [[564  52   1   0]
[350 617  61   6]
[  8  86 568   9]
[  0   0  11  66]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7471
   Precision: 0.7548
   Recall:    0.7471
   F1 Score:  0.7480

 Confusion Matrix:
   [[443 137   0   0]
[110 474  26   0]
[ 10  59 111   6]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Logistic Regression...

 Results  (n_test=2,321):
   Accuracy:  0.7238
   Precision: 0.7236
   Recall:    0.7238
   F1 Score:  0.7211

 Confusion Matrix:
   [[711 127   1   2]
[246 517  53   3]
[ 11 138 437  21]
[  0   0  39  15]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training Logistic Regression...
```

```
  Results  (n_test=2,490):
    Accuracy:  0.7386
    Precision: 0.7393
    Recall:    0.7386
    F1 Score:  0.7294

  Confusion Matrix:
    [[740  61   3   2]
 [235 629  50   3]
 [  7 214 454   3]
 [  1  20  52  16]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: GDELT only
======================================================================
Using 3485 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.6144
    Precision: 0.6367
    Recall:    0.6144
    F1 Score:  0.6077

  Confusion Matrix:
    [[417  63   0   0]
 [366 409  54   1]
 [  2 154 290  22]
 [  0   3  75  63]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Logistic Regression...

  Results  (n_test=2,313):
    Accuracy:  0.7735
    Precision: 0.7747
    Recall:    0.7735
    F1 Score:  0.7718

  Confusion Matrix:
    [[773 150   1   0]
 [110 609  89   0]
 [  0  81 367  24]
 [  0   4  65  40]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training Logistic Regression...
```

```
 Results  (n_test=982):
   Accuracy:  0.5978
   Precision: 0.7400
   Recall:    0.5978
   F1 Score:  0.6152

 Confusion Matrix:
   [[303 299   1   0]
[ 26 206  39   1]
[  2  25  76   0]
[  0   0   2   2]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7574
   Precision: 0.7847
   Recall:    0.7574
   F1 Score:  0.7558

 Confusion Matrix:
   [[564  52   1   0]
[350 619  58   7]
[  8  86 567  10]
[  0   0  10  67]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7464
   Precision: 0.7568
   Recall:    0.7464
   F1 Score:  0.7470

 Confusion Matrix:
   [[429 151   0   0]
[ 94 489  27   0]
[  8  64 109   5]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Logistic Regression...

 Results  (n_test=2,321):
   Accuracy:  0.7264
   Precision: 0.7226
   Recall:    0.7264
   F1 Score:  0.7218
```

```
  Confusion Matrix:
    [[711 128   1   1]
 [249 511  56   3]
 [  9 130 454  14]
 [  0   0  44  10]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Logistic Regression...

  Results  (n_test=2,490):
    Accuracy:  0.7257
    Precision: 0.7255
    Recall:    0.7257
    F1 Score:  0.7148

  Confusion Matrix:
    [[746  55   3   2]
 [268 596  50   3]
 [  8 215 452   3]
 [  0  20  56  13]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC only (no geo)
======================================================================
Using 1 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.5076
    Precision: 0.4937
    Recall:    0.5076
    F1 Score:  0.4664

  Confusion Matrix:
    [[431  48   1   0]
 [516 254  60   0]
 [ 23 156 289   0]
 [  0   4 137   0]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Logistic Regression...

  Results  (n_test=2,313):
    Accuracy:  0.6667
    Precision: 0.6794
    Recall:    0.6667
    F1 Score:  0.6560
```

```
 Confusion Matrix:
   [[572 350   2   0]
[ 62 603 143   0]
[  2 103 367   0]
[  0   1 108   0]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Logistic Regression...

 Results  (n_test=982):
   Accuracy:  0.6599
   Precision: 0.7151
   Recall:    0.6599
   F1 Score:  0.6742

 Confusion Matrix:
   [[398 204   1   0]
[ 72 177  23   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7049
   Precision: 0.6941
   Recall:    0.7049
   F1 Score:  0.6907

 Confusion Matrix:
   [[512 101   4   0]
[321 618  95   0]
[  9 101 561   0]
[  0   0  77   0]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7478
   Precision: 0.7713
   Recall:    0.7478
   F1 Score:  0.7483

 Confusion Matrix:
   [[392 188   0]
[ 37 519  54]
[  7  61 118]]
```

```
  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Logistic Regression...

  Results  (n_test=2,321):
    Accuracy:  0.7299
    Precision: 0.7103
    Recall:    0.7299
    F1 Score:  0.7196

  Confusion Matrix:
    [[643 198   0   0]
 [233 520  66   0]
 [  4  72 531   0]
 [  0   0  54   0]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Logistic Regression...

  Results  (n_test=2,490):
    Accuracy:  0.7301
    Precision: 0.7087
    Recall:    0.7301
    F1 Score:  0.7108

  Confusion Matrix:
    [[782  24   0   0]
 [292 592  33   0]
 [ 12 222 444   0]
 [  0  21  68   0]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC + GDELT base (no geo)
======================================================================
Using 120 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.5070
    Precision: 0.5417
    Recall:    0.5070
    F1 Score:  0.4673

  Confusion Matrix:
    [[430  49   1   0]
 [516 253  61   0]
 [ 23 156 288   1]
 [  0   4 135   2]]
```

```
 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Logistic Regression...

 Results  (n_test=2,313):
   Accuracy:  0.6671
   Precision: 0.7080
   Recall:    0.6671
   F1 Score:  0.6584

 Confusion Matrix:
   [[572 350   2   0]
[ 62 604 142   0]
[  2 104 364   2]
[  0   1 105   3]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Logistic Regression...

 Results  (n_test=982):
   Accuracy:  0.6609
   Precision: 0.7176
   Recall:    0.6609
   F1 Score:  0.6753

 Confusion Matrix:
   [[397 205   1   0]
[ 70 179  23   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7049
   Precision: 0.7185
   Recall:    0.7049
   F1 Score:  0.6922

 Confusion Matrix:
   [[511 100   6   0]
[319 614 101   0]
[ 10  97 563   1]
[  0   0  74   3]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...
```

```
  Results  (n_test=1,376):
    Accuracy:  0.7449
    Precision: 0.7690
    Recall:    0.7449
    F1 Score:  0.7457

  Confusion Matrix:
    [[391 188   1   0]
 [ 37 517  56   0]
 [  7  61 117   1]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Logistic Regression...

  Results  (n_test=2,321):
    Accuracy:  0.7286
    Precision: 0.7213
    Recall:    0.7286
    F1 Score:  0.7199

  Confusion Matrix:
    [[641 200   0   0]
 [233 520  66   0]
 [  4  73 528   2]
 [  0   0  52   2]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Logistic Regression...

  Results  (n_test=2,490):
    Accuracy:  0.7273
    Precision: 0.7242
    Recall:    0.7273
    F1 Score:  0.7087

  Confusion Matrix:
    [[780  26   0   0]
 [293 590  34   0]
 [ 13 224 440   1]
 [  0  21  67   1]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: IPC + GDELT + temporal (no geo)
======================================================================
Using 155 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...
```

```
 Results  (n_test=1,919):
   Accuracy:  0.5081
   Precision: 0.5662
   Recall:    0.5081
   F1 Score:  0.4706

 Confusion Matrix:
   [[427  52   1   0]
[515 254  61   0]
[ 24 155 289   0]
[  0   4 132   5]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Logistic Regression...

 Results  (n_test=2,313):
   Accuracy:  0.6667
   Precision: 0.7090
   Recall:    0.6667
   F1 Score:  0.6601

 Confusion Matrix:
   [[570 352   2   0]
[ 63 602 143   0]
[  2 103 363   4]
[  0   1 101   7]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Logistic Regression...

 Results  (n_test=982):
   Accuracy:  0.6568
   Precision: 0.7144
   Recall:    0.6568
   F1 Score:  0.6715

 Confusion Matrix:
   [[395 207   1   0]
[ 70 177  25   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.7045
   Precision: 0.7255
   Recall:    0.7045
```

```
  F1 Score:  0.6926

 Confusion Matrix:
   [[511 101   5   0]
[321 611 102   0]
[ 10  98 563   0]
[  0   0  72   5]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.7398
   Precision: 0.7646
   Recall:    0.7398
   F1 Score:  0.7414

 Confusion Matrix:
   [[391 188   1   0]
[ 38 512  60   0]
[  7  61 115   3]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Logistic Regression...

 Results  (n_test=2,321):
   Accuracy:  0.7264
   Precision: 0.7149
   Recall:    0.7264
   F1 Score:  0.7168

 Confusion Matrix:
   [[641 198   2   0]
[233 514  72   0]
[  4  71 530   2]
[  0   0  53   1]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training Logistic Regression...

 Results  (n_test=2,490):
   Accuracy:  0.7261
   Precision: 0.7169
   Recall:    0.7261
   F1 Score:  0.7075

 Confusion Matrix:
   [[779  27   0   0]
[295 585  37   0]
```

```
 [ 13 220 443   2]
 [  0  20  68   1]]

======================================================================
MODEL: Logistic Regression  |  FEATURE SET: GDELT only (no geo)
======================================================================
Using 154 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Logistic Regression...

  Results  (n_test=1,919):
    Accuracy:  0.2939
    Precision: 0.3954
    Recall:    0.2939
    F1 Score:  0.2280

  Confusion Matrix:
    [[407  57  16   0]
 [695 113  22   0]
 [350  74  42   2]
 [107  12  20   2]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Logistic Regression...

  Results  (n_test=2,313):
    Accuracy:  0.4289
    Precision: 0.4343
    Recall:    0.4289
    F1 Score:  0.3511

  Confusion Matrix:
    [[805  98  21   0]
 [652 120  36   0]
 [341  64  64   3]
 [ 82   7  17   3]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training Logistic Regression...

  Results  (n_test=982):
    Accuracy:  0.2811
    Precision: 0.4333
    Recall:    0.2811
    F1 Score:  0.1724

  Confusion Matrix:
    [[ 23 566  14   0]
 [ 20 244   8   0]
```

```
[  0  94   9   0]
[  0   4   0   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Logistic Regression...

 Results  (n_test=2,399):
   Accuracy:  0.2868
   Precision: 0.4015
   Recall:    0.2868
   F1 Score:  0.2017

 Confusion Matrix:
   [[551  42  24   0]
[906  79  49   0]
[564  52  55   0]
[ 58   8   8   3]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Logistic Regression...

 Results  (n_test=1,376):
   Accuracy:  0.4397
   Precision: 0.4209
   Recall:    0.4397
   F1 Score:  0.3477

 Confusion Matrix:
   [[522  48  10   0]
[509  72  28   1]
[136  38  11   1]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Logistic Regression...

 Results  (n_test=2,321):
   Accuracy:  0.3800
   Precision: 0.3954
   Recall:    0.3800
   F1 Score:  0.2801

 Confusion Matrix:
   [[758  67  15   1]
[715  80  24   0]
[504  57  44   2]
[ 37   9   8   0]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
```

```
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Logistic Regression...

  Results  (n_test=2,490):
    Accuracy:  0.3434
    Precision: 0.3840
    Recall:    0.3434
    F1 Score:  0.2337

  Confusion Matrix:
    [[748  46  12   0]
 [828  62  27   0]
 [578  55  44   1]
 [ 66   8  14   1]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC only
======================================================================
Using 3332 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5628
    Precision: 0.6280
    Recall:    0.5628
    F1 Score:  0.5350

  Confusion Matrix:
    [[476   0   2   2]
 [430 330  55  15]
 [  6 142 145 175]
 [  0   2  10 129]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Random Forest...

  Results  (n_test=2,313):
    Accuracy:  0.6559
    Precision: 0.7040
    Recall:    0.6559
    F1 Score:  0.6662

  Confusion Matrix:
    [[676 235   2  11]
 [114 551  91  52]
 [  5  88 198 181]
 [  0   0  17  92]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
```

```
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.4613
   Precision: 0.6990
   Recall:    0.4613
   F1 Score:  0.4481

 Confusion Matrix:
   [[156 446   1   0]
[ 24 225  23   0]
[  0  31  71   1]
[  0   0   3   1]]

 —— Eval 4: predict at 202210 → target 202306 ——
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.6640
   Precision: 0.7273
   Recall:    0.6640
   F1 Score:  0.6706

 Confusion Matrix:
   [[524  83   0  10]
[378 577  58  21]
[ 34  94 420 123]
[  0   0   5  72]]

 —— Eval 5: predict at 202206 → target 202302 ——
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.7304
   Precision: 0.7752
   Recall:    0.7304
   F1 Score:  0.7253

 Confusion Matrix:
   [[389 191   0   0]
[ 35 552  17   6]
[ 10  99  64  13]
[  0   0   0   0]]

 —— Eval 6: predict at 202202 → target 202210 ——
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Random Forest...

 Results  (n_test=2,321):
   Accuracy:  0.6695
```

```
    Precision: 0.7051
    Recall:    0.6695
    F1 Score:  0.6778

  Confusion Matrix:
    [[645 194   0   2]
 [282 492  23  22]
 [ 20 113 382  92]
 [  0   0  19  35]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.6960
    Precision: 0.7152
    Recall:    0.6960
    F1 Score:  0.6876

  Confusion Matrix:
    [[768  37   0   1]
 [335 556  24   2]
 [ 47 205 371  55]
 [  0   9  42  38]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC + GDELT base
======================================================================
Using 3451 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5758
    Precision: 0.6340
    Recall:    0.5758
    F1 Score:  0.5532

  Confusion Matrix:
    [[470   4   4   2]
 [403 356  57  14]
 [ 13 139 156 160]
 [  1   1  16 123]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Random Forest...

  Results  (n_test=2,313):
    Accuracy:  0.6576
```

```
  Precision: 0.6963
  Recall:    0.6576
  F1 Score:  0.6678

 Confusion Matrix:
   [[689 221   4  10]
[128 556  77  47]
[  5 101 207 159]
[  0   0  40  69]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.4939
   Precision: 0.6783
   Recall:    0.4939
   F1 Score:  0.5020

 Confusion Matrix:
   [[213 390   0   0]
[ 42 206  22   2]
[  0  37  66   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.6553
   Precision: 0.7164
   Recall:    0.6553
   F1 Score:  0.6614

 Confusion Matrix:
   [[515  89   0  13]
[370 583  54  27]
[ 30 123 399 119]
[  0   0   2  75]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.7333
   Precision: 0.7731
   Recall:    0.7333
   F1 Score:  0.7325

 Confusion Matrix:
```

```
    [[398 179   2   1]
 [ 35 542  24   9]
 [ 10  90  69  17]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Random Forest...

  Results  (n_test=2,321):
    Accuracy:  0.6704
    Precision: 0.7030
    Recall:    0.6704
    F1 Score:  0.6782

  Confusion Matrix:
    [[643 197   1   0]
[276 498  26  19]
[ 18 116 382  91]
[  0   0  21  33]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.6880
    Precision: 0.7072
    Recall:    0.6880
    F1 Score:  0.6781

  Confusion Matrix:
    [[777  29   0   0]
 [347 547  23   0]
 [ 52 202 357  67]
 [  0   2  55  32]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC + GDELT + temporal
======================================================================
Using 3486 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5816
    Precision: 0.6350
    Recall:    0.5816
    F1 Score:  0.5627

  Confusion Matrix:
```

```
   [[466   6   5   3]
[404 363  56   7]
[ 21 136 174 137]
[  0   2  26 113]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Random Forest...

 Results  (n_test=2,313):
   Accuracy:  0.6632
   Precision: 0.6969
   Recall:    0.6632
   F1 Score:  0.6741

 Confusion Matrix:
   [[692 223   4   5]
[125 534 105  44]
[  5  81 239 147]
[  0   0  40  69]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.5295
   Precision: 0.6977
   Recall:    0.5295
   F1 Score:  0.5400

 Confusion Matrix:
   [[240 362   1   0]
[ 40 209  23   0]
[  0  31  71   1]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.6661
   Precision: 0.7226
   Recall:    0.6661
   F1 Score:  0.6719

 Confusion Matrix:
   [[518  86   4   9]
[376 581  58  19]
[ 25 103 427 116]
[  0   0   5  72]]
```

```
  ── Eval 5: predict at 202206 → target 202302 ──
  Training samples:   13,794
  Test samples:       1,376  (areas in both 202206 and 202302)
  Training Random Forest...

  Results  (n_test=1,376):
    Accuracy:  0.7362
    Precision: 0.7761
    Recall:    0.7362
    F1 Score:  0.7353

  Confusion Matrix:
    [[402 176   0   2]
 [ 37 542  23   8]
 [ 10  90  69  17]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Random Forest...

  Results  (n_test=2,321):
    Accuracy:  0.6764
    Precision: 0.7052
    Recall:    0.6764
    F1 Score:  0.6838

  Confusion Matrix:
    [[650 189   2   0]
 [271 498  29  21]
 [ 16 114 392  85]
 [  0   0  24  30]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.6827
    Precision: 0.6981
    Recall:    0.6827
    F1 Score:  0.6728

  Confusion Matrix:
    [[773  32   0   1]
 [353 534  29   1]
 [ 46 197 369  66]
 [  0   4  61  24]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: GDELT only
======================================================================
Using 3485 features
```

```
 ── Eval 1: predict at 202310 → target 202406 ──
 Training samples:   11,873
 Test samples:       1,919  (areas in both 202310 and 202406)
 Training Random Forest...

 Results  (n_test=1,919):
   Accuracy:  0.5466
   Precision: 0.5668
   Recall:    0.5466
   F1 Score:  0.5221

 Confusion Matrix:
   [[413  25  18  24]
[337 428  52  13]
[ 30 195  95 148]
[  0   4  24 113]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Random Forest...

 Results  (n_test=2,313):
   Accuracy:  0.6126
   Precision: 0.6422
   Recall:    0.6126
   F1 Score:  0.6151

 Confusion Matrix:
   [[667 217  15  25]
[185 532  43  48]
[ 47 126 156 143]
[  0   4  43  62]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.4124
   Precision: 0.6281
   Recall:    0.4124
   F1 Score:  0.3784

 Confusion Matrix:
   [[141 454   8   0]
[ 19 248   5   0]
[ 20  64  16   3]
[  0   4   0   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...
```

```
 Results  (n_test=2,399):
   Accuracy:  0.5969
   Precision: 0.6882
   Recall:    0.5969
   F1 Score:  0.6059

 Confusion Matrix:
   [[462 123   6  26]
[409 581  19  25]
[110 130 317 114]
[  0   4   1  72]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.5451
   Precision: 0.5858
   Recall:    0.5451
   F1 Score:  0.5129

 Confusion Matrix:
   [[237 341   0   2]
[ 91 497   9  13]
[ 47 108  16  15]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Random Forest...

 Results  (n_test=2,321):
   Accuracy:  0.6295
   Precision: 0.6814
   Recall:    0.6295
   F1 Score:  0.6387

 Confusion Matrix:
   [[627 183   9  22]
[286 488  12  33]
[ 66 136 317  88]
[  8   1  16  29]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training Random Forest...

 Results  (n_test=2,490):
   Accuracy:  0.6490
   Precision: 0.6759
   Recall:    0.6490
   F1 Score:  0.6430
```

```
  Confusion Matrix:
    [[658 134   5   9]
 [260 638  10   9]
 [ 93 213 292  80]
 [  5   2  54  28]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC only (no geo)
======================================================================
Using 1 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5221
    Precision: 0.5592
    Recall:    0.5221
    F1 Score:  0.4995

  Confusion Matrix:
    [[431  48   1   0]
 [516 254  60   0]
 [ 23 156 274  15]
 [  0   4  94  43]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Random Forest...

  Results  (n_test=2,313):
    Accuracy:  0.6671
    Precision: 0.7073
    Recall:    0.6671
    F1 Score:  0.6725

  Confusion Matrix:
    [[572 350   2   0]
 [ 62 603 143   0]
 [  2 103 314  53]
 [  0   1  54  54]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training Random Forest...

  Results  (n_test=982):
    Accuracy:  0.6599
    Precision: 0.7151
    Recall:    0.6599
    F1 Score:  0.6742
```

```
 Confusion Matrix:
   [[398 204   1   0]
[ 72 177  23   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.7011
   Precision: 0.7165
   Recall:    0.7011
   F1 Score:  0.7004

 Confusion Matrix:
   [[512 101   4   0]
[321 618  94   1]
[  9 101 513  48]
[  0   0  38  39]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.7442
   Precision: 0.7700
   Recall:    0.7442
   F1 Score:  0.7457

 Confusion Matrix:
   [[392 188   0   0]
[ 37 519  54   0]
[  7  61 113   5]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Random Forest...

 Results  (n_test=2,321):
   Accuracy:  0.7281
   Precision: 0.7240
   Recall:    0.7281
   F1 Score:  0.7254

 Confusion Matrix:
   [[643 198   0   0]
[233 520  65   1]
[  4  72 513  18]
```

```
 [  0   0  40  14]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.7341
    Precision: 0.7460
    Recall:    0.7341
    F1 Score:  0.7201

  Confusion Matrix:
    [[782  24   0   0]
 [292 592  33   0]
 [ 12 222 443   1]
 [  0  21  57  11]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC + GDELT base (no geo)
======================================================================
Using 120 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5216
    Precision: 0.5582
    Recall:    0.5216
    F1 Score:  0.5019

  Confusion Matrix:
    [[419  60   1   0]
 [506 264  60   0]
 [ 24 155 275  14]
 [  0   4  94  43]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training Random Forest...

  Results  (n_test=2,313):
    Accuracy:  0.6645
    Precision: 0.7057
    Recall:    0.6645
    F1 Score:  0.6693

  Confusion Matrix:
    [[559 360   4   1]
 [ 64 603 141   0]
 [  2 101 322  47]
```

```
[  0   1  55  53]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.6497
   Precision: 0.7100
   Recall:    0.6497
   F1 Score:  0.6649

 Confusion Matrix:
   [[389 213   1   0]
[ 71 176  25   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.6978
   Precision: 0.7123
   Recall:    0.6978
   F1 Score:  0.6975

 Confusion Matrix:
   [[501 111   5   0]
[317 620  94   3]
[  9  98 515  49]
[  0   0  39  38]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.7376
   Precision: 0.7648
   Recall:    0.7376
   F1 Score:  0.7395

 Confusion Matrix:
   [[388 191   1   0]
[ 36 515  59   0]
[  7  62 112   5]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
```

```
  Training Random Forest...

  Results  (n_test=2,321):
    Accuracy:  0.7264
    Precision: 0.7216
    Recall:    0.7264
    F1 Score:  0.7233

  Confusion Matrix:
    [[639 202   0   0]
 [228 522  68   1]
 [  4  72 513  18]
 [  0   0  42  12]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.7237
    Precision: 0.7356
    Recall:    0.7237
    F1 Score:  0.7095

  Confusion Matrix:
    [[772  34   0   0]
 [302 577  38   0]
 [ 15 219 443   1]
 [  0  22  57  10]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: IPC + GDELT + temporal (no geo)
======================================================================
Using 155 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.5190
    Precision: 0.5520
    Recall:    0.5190
    F1 Score:  0.4980

  Confusion Matrix:
    [[422  55   2   1]
 [506 264  60   0]
 [ 23 156 270  19]
 [  0   3  98  40]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
```

```
 Training Random Forest...

 Results  (n_test=2,313):
   Accuracy:  0.6610
   Precision: 0.7028
   Recall:    0.6610
   F1 Score:  0.6654

 Confusion Matrix:
   [[555 363   6   0]
[ 63 597 147   1]
[  2  97 330  43]
[  0   1  61  47]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.6446
   Precision: 0.7062
   Recall:    0.6446
   F1 Score:  0.6598

 Confusion Matrix:
   [[385 214   4   0]
[ 70 175  27   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.6965
   Precision: 0.7106
   Recall:    0.6965
   F1 Score:  0.6960

 Confusion Matrix:
   [[503 106   8   0]
[315 620  96   3]
[  9 100 513  49]
[  0   0  42  35]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.7369
   Precision: 0.7623
```

```
    Recall:    0.7369
    F1 Score:  0.7388

  Confusion Matrix:
    [[388 188   4   0]
 [ 36 505  69   0]
 [  7  57 121   1]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training Random Forest...

  Results  (n_test=2,321):
    Accuracy:  0.7199
    Precision: 0.7143
    Recall:    0.7199
    F1 Score:  0.7163

  Confusion Matrix:
    [[639 198   4   0]
 [233 507  76   3]
 [  4  72 513  18]
 [  0   0  42  12]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training Random Forest...

  Results  (n_test=2,490):
    Accuracy:  0.7261
    Precision: 0.7319
    Recall:    0.7261
    F1 Score:  0.7115

  Confusion Matrix:
    [[774  32   0   0]
 [297 579  41   0]
 [ 15 214 446   3]
 [  0  20  60   9]]

======================================================================
MODEL: Random Forest  |  FEATURE SET: GDELT only (no geo)
======================================================================
Using 154 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training Random Forest...

  Results  (n_test=1,919):
    Accuracy:  0.3163
    Precision: 0.4356
```

```
  Recall:    0.3163
  F1 Score:  0.2638

 Confusion Matrix:
   [[403  33  37   7]
[655 102  62  11]
[322  40  94  12]
[109   6  18   8]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training Random Forest...

 Results  (n_test=2,313):
   Accuracy:  0.4371
   Precision: 0.4542
   Recall:    0.4371
   F1 Score:  0.3670

 Confusion Matrix:
   [[793  67  60   4]
[620 105  79   4]
[332  31 105   4]
[ 81   0  20   8]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training Random Forest...

 Results  (n_test=982):
   Accuracy:  0.5845
   Precision: 0.5050
   Recall:    0.5845
   F1 Score:  0.5086

 Confusion Matrix:
   [[537  40  21   5]
[225  18  23   6]
[ 82   1  19   1]
[  4   0   0   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training Random Forest...

 Results  (n_test=2,399):
   Accuracy:  0.2939
   Precision: 0.4204
   Recall:    0.2939
   F1 Score:  0.2156

 Confusion Matrix:
   [[543  33  38   3]
```

```
[882  70  72  10]
[551  25  85  10]
[ 53   3  14   7]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training Random Forest...

 Results  (n_test=1,376):
   Accuracy:  0.4339
   Precision: 0.4419
   Recall:    0.4339
   F1 Score:  0.3359

 Confusion Matrix:
   [[523  31  23   3]
[509  46  52   3]
[139  17  28   2]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training Random Forest...

 Results  (n_test=2,321):
   Accuracy:  0.3882
   Precision: 0.4210
   Recall:    0.3882
   F1 Score:  0.2949

 Confusion Matrix:
   [[756  38  45   2]
[689  64  56  10]
[496  27  77   7]
[ 36   2  12   4]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training Random Forest...

 Results  (n_test=2,490):
   Accuracy:  0.3438
   Precision: 0.3781
   Recall:    0.3438
   F1 Score:  0.2581

 Confusion Matrix:
   [[698  62  40   6]
[782  77  55   3]
[555  33  80  10]
[ 61  13  14   1]]

======================================================================
```

```
MODEL: CatBoost  |  FEATURE SET: IPC only
======================================================================
Using 3332 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5393
    Precision: 0.5900
    Recall:    0.5393
    F1 Score:  0.5190

  Confusion Matrix:
    [[431  48   1   0]
 [499 271  49  11]
 [ 23 143 213  89]
 [  0   2  19 120]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.6740
    Precision: 0.7146
    Recall:    0.6740
    F1 Score:  0.6849

  Confusion Matrix:
    [[651 267   6   0]
 [ 67 581 137  23]
 [  2  80 253 137]
 [  0   0  35  74]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training CatBoost...

  Results  (n_test=982):
    Accuracy:  0.6426
    Precision: 0.7162
    Recall:    0.6426
    F1 Score:  0.6600

  Confusion Matrix:
    [[375 227   1   0]
 [ 67 182  23   0]
 [  0  30  70   3]
 [  0   0   0   4]]

  ── Eval 4: predict at 202210 → target 202306 ──
```

```
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.6895
   Precision: 0.7243
   Recall:    0.6895
   F1 Score:  0.6925

 Confusion Matrix:
   [[512 100   4   1]
[322 609  93  10]
[  9  94 461 107]
[  0   0   5  72]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
   Accuracy:  0.7427
   Precision: 0.7706
   Recall:    0.7427
   F1 Score:  0.7452

 Confusion Matrix:
   [[392 188   0   0]
[ 37 519  52   2]
[  7  61 111   7]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training CatBoost...

 Results  (n_test=2,321):
   Accuracy:  0.6945
   Precision: 0.7155
   Recall:    0.6945
   F1 Score:  0.7008

 Confusion Matrix:
   [[643 198   0   0]
[233 521  55  10]
[  4 105 414  84]
[  0   0  20  34]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training CatBoost...

 Results  (n_test=2,490):
```

```
    Accuracy:  0.7249
    Precision: 0.7306
    Recall:    0.7249
    F1 Score:  0.7166

  Confusion Matrix:
    [[773  33   0   0]
 [294 582  41   0]
 [ 12 221 417  28]
 [  0  15  41  33]]

================================================================================
MODEL: CatBoost  |  FEATURE SET: IPC + GDELT base
================================================================================
Using 3451 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5393
    Precision: 0.5881
    Recall:    0.5393
    F1 Score:  0.5195

  Confusion Matrix:
    [[430  49   1   0]
 [499 271  49  11]
 [ 22 145 216  85]
 [  0   2  21 118]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.6714
    Precision: 0.7151
    Recall:    0.6714
    F1 Score:  0.6823

  Confusion Matrix:
    [[635 283   6   0]
 [ 63 587 133  25]
 [  2  80 256 134]
 [  0   0  34  75]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training CatBoost...

  Results  (n_test=982):
```

```
  Accuracy:  0.6436
  Precision: 0.7246
  Recall:    0.6436
  F1 Score:  0.6612

 Confusion Matrix:
   [[368 234   1   0]
[ 60 189  23   0]
[  0  30  71   2]
[  0   0   0   4]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
  Accuracy:  0.6965
  Precision: 0.7310
  Recall:    0.6965
  F1 Score:  0.7005

 Confusion Matrix:
   [[512 101   3   1]
[300 635  89  10]
[  9  97 452 113]
[  0   0   5  72]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
  Accuracy:  0.7384
  Precision: 0.7696
  Recall:    0.7384
  F1 Score:  0.7423

 Confusion Matrix:
   [[392 188   0   0]
[ 37 518  51   4]
[  7  61 106  12]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training CatBoost...

 Results  (n_test=2,321):
  Accuracy:  0.6941
  Precision: 0.7148
  Recall:    0.6941
  F1 Score:  0.7003
```

```
  Confusion Matrix:
    [[643 198   0   0]
 [232 521  57   9]
 [  4 105 413  85]
 [  0   0  20  34]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.7301
    Precision: 0.7352
    Recall:    0.7301
    F1 Score:  0.7213

  Confusion Matrix:
    [[781  25   0   0]
 [287 589  41   0]
 [ 12 222 416  28]
 [  0  16  41  32]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: IPC + GDELT + temporal
======================================================================
Using 3486 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5451
    Precision: 0.5920
    Recall:    0.5451
    F1 Score:  0.5259

  Confusion Matrix:
    [[430  49   1   0]
 [496 274  51   9]
 [ 21 146 223  78]
 [  0   2  20 119]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.6796
    Precision: 0.7209
    Recall:    0.6796
    F1 Score:  0.6898
```

```
 Confusion Matrix:
   [[645 277   2   0]
[ 65 590 128  25]
[  2  80 260 130]
[  0   0  32  77]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training CatBoost...

 Results  (n_test=982):
   Accuracy:  0.6354
   Precision: 0.7121
   Recall:    0.6354
   F1 Score:  0.6533

 Confusion Matrix:
   [[368 234   1   0]
[ 68 181  23   0]
[  0  30  71   2]
[  0   0   0   4]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.6961
   Precision: 0.7304
   Recall:    0.6961
   F1 Score:  0.7000

 Confusion Matrix:
   [[512 100   4   1]
[299 636  89  10]
[  9  99 450 113]
[  0   0   5  72]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
   Accuracy:  0.7398
   Precision: 0.7681
   Recall:    0.7398
   F1 Score:  0.7425

 Confusion Matrix:
   [[391 188   1   0]
[ 37 518  54   1]
[  7  61 109   9]
[  0   0   0   0]]
```

```
  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training CatBoost...

  Results  (n_test=2,321):
    Accuracy:  0.6963
    Precision: 0.7150
    Recall:    0.6963
    F1 Score:  0.7018

  Confusion Matrix:
    [[644 197   0   0]
 [235 517  59   8]
 [  4 102 420  81]
 [  0   0  19  35]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.7269
    Precision: 0.7331
    Recall:    0.7269
    F1 Score:  0.7186

  Confusion Matrix:
    [[774  32   0   0]
 [295 584  38   0]
 [ 12 220 420  26]
 [  0  16  41  32]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: GDELT only
======================================================================
Using 3485 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5883
    Precision: 0.6169
    Recall:    0.5883
    F1 Score:  0.5691

  Confusion Matrix:
    [[431  30  10   9]
 [334 425  53  18]
 [ 21 190 148 109]
 [  1   0  15 125]]
```

```
 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training CatBoost...

 Results  (n_test=2,313):
   Accuracy:  0.6580
   Precision: 0.6845
   Recall:    0.6580
   F1 Score:  0.6617

 Confusion Matrix:
   [[704 203   8   9]
[176 538  60  34]
[ 15 115 203 139]
[  0   0  32  77]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training CatBoost...

 Results  (n_test=982):
   Accuracy:  0.5112
   Precision: 0.6590
   Recall:    0.5112
   F1 Score:  0.5294

 Confusion Matrix:
   [[266 330   7   0]
[ 47 189  35   1]
[  6  51  43   3]
[  0   0   0   4]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.6649
   Precision: 0.7267
   Recall:    0.6649
   F1 Score:  0.6721

 Confusion Matrix:
   [[507  91  10   9]
[378 604  39  13]
[ 42 103 410 116]
[  0   3   0  74]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...
```

```
  Results  (n_test=1,376):
    Accuracy:  0.6686
    Precision: 0.6890
    Recall:    0.6686
    F1 Score:  0.6634

  Confusion Matrix:
    [[384 191   5   0]
 [ 99 481  20  10]
 [ 30  88  55  13]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training CatBoost...

  Results  (n_test=2,321):
    Accuracy:  0.6467
    Precision: 0.6844
    Recall:    0.6467
    F1 Score:  0.6494

  Confusion Matrix:
    [[713 105  12  11]
 [324 425  47  23]
 [ 32 131 328 116]
 [  0   1  18  35]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.6498
    Precision: 0.6694
    Recall:    0.6498
    F1 Score:  0.6398

  Confusion Matrix:
    [[747  55   4   0]
 [381 496  34   6]
 [ 54 215 340  69]
 [  2   0  52  35]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: IPC only (no geo)
======================================================================
Using 1 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...
```

```
 Results  (n_test=1,919):
   Accuracy:  0.5221
   Precision: 0.5592
   Recall:    0.5221
   F1 Score:  0.4995

 Confusion Matrix:
   [[431  48   1   0]
[516 254  60   0]
[ 23 156 274  15]
[  0   4  94  43]]

 ── Eval 2: predict at 202306 → target 202402 ──
 Training samples:   12,442
 Test samples:       2,313  (areas in both 202306 and 202402)
 Training CatBoost...

 Results  (n_test=2,313):
   Accuracy:  0.6671
   Precision: 0.7073
   Recall:    0.6671
   F1 Score:  0.6725

 Confusion Matrix:
   [[572 350   2   0]
[ 62 603 143   0]
[  2 103 314  53]
[  0   1  54  54]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training CatBoost...

 Results  (n_test=982):
   Accuracy:  0.6599
   Precision: 0.7151
   Recall:    0.6599
   F1 Score:  0.6742

 Confusion Matrix:
   [[398 204   1   0]
[ 72 177  23   0]
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.7011
   Precision: 0.7165
   Recall:    0.7011
```

```
  F1 Score:  0.7004

 Confusion Matrix:
   [[512 101   4   0]
[321 618  94   1]
[  9 101 513  48]
[  0   0  38  39]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
   Accuracy:  0.7442
   Precision: 0.7700
   Recall:    0.7442
   F1 Score:  0.7457

 Confusion Matrix:
   [[392 188   0   0]
[ 37 519  54   0]
[  7  61 113   5]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training CatBoost...

 Results  (n_test=2,321):
   Accuracy:  0.7281
   Precision: 0.7240
   Recall:    0.7281
   F1 Score:  0.7254

 Confusion Matrix:
   [[643 198   0   0]
[233 520  65   1]
[  4  72 513  18]
[  0   0  40  14]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
 Test samples:       2,490  (areas in both 202110 and 202206)
 Training CatBoost...

 Results  (n_test=2,490):
   Accuracy:  0.7341
   Precision: 0.7460
   Recall:    0.7341
   F1 Score:  0.7201

 Confusion Matrix:
   [[782  24   0   0]
[292 592  33   0]
```

```
 [ 12 222 443   1]
 [  0  21  57  11]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: IPC + GDELT base (no geo)
======================================================================
Using 120 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5211
    Precision: 0.5558
    Recall:    0.5211
    F1 Score:  0.4985

  Confusion Matrix:
    [[431  47   2   0]
 [513 257  60   0]
 [ 23 156 270  19]
 [  0   4  95  42]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.6675
    Precision: 0.7074
    Recall:    0.6675
    F1 Score:  0.6727

  Confusion Matrix:
    [[571 350   3   0]
 [ 62 601 145   0]
 [  2 100 320  50]
 [  0   1  56  52]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
  Test samples:       982  (areas in both 202302 and 202310)
  Training CatBoost...

  Results  (n_test=982):
    Accuracy:  0.6589
    Precision: 0.7156
    Recall:    0.6589
    F1 Score:  0.6733

  Confusion Matrix:
    [[396 206   1   0]
 [ 71 178  23   0]
```

```
[  0  30  73   0]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.7003
   Precision: 0.7161
   Recall:    0.7003
   F1 Score:  0.6999

 Confusion Matrix:
   [[509 102   6   0]
[318 619  95   2]
[  9  99 512  51]
[  0   0  37  40]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
   Accuracy:  0.7427
   Precision: 0.7690
   Recall:    0.7427
   F1 Score:  0.7447

 Confusion Matrix:
   [[392 188   0   0]
[ 37 517  56   0]
[  7  60 113   6]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training CatBoost...

 Results  (n_test=2,321):
   Accuracy:  0.7264
   Precision: 0.7223
   Recall:    0.7264
   F1 Score:  0.7238

 Confusion Matrix:
   [[641 199   1   0]
[231 520  67   1]
[  4  72 511  20]
[  0   0  40  14]]

 ── Eval 7: predict at 202110 → target 202206 ──
 Training samples:   12,948
```

```
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.7301
    Precision: 0.7398
    Recall:    0.7301
    F1 Score:  0.7162

  Confusion Matrix:
    [[779  27   0   0]
 [293 590  34   0]
 [ 12 226 438   2]
 [  0  21  57  11]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: IPC + GDELT + temporal (no geo)
======================================================================
Using 155 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.5190
    Precision: 0.5524
    Recall:    0.5190
    F1 Score:  0.4963

  Confusion Matrix:
    [[431  48   1   0]
 [517 254  58   1]
 [ 23 154 268  23]
 [  0   3  95  43]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.6693
    Precision: 0.7087
    Recall:    0.6693
    F1 Score:  0.6745

  Confusion Matrix:
    [[572 349   3   0]
 [ 62 600 145   1]
 [  2  95 323  52]
 [  0   0  56  53]]

  ── Eval 3: predict at 202302 → target 202310 ──
  Training samples:   13,462
```

```
 Test samples:       982  (areas in both 202302 and 202310)
 Training CatBoost...

 Results  (n_test=982):
   Accuracy:  0.6538
   Precision: 0.7118
   Recall:    0.6538
   F1 Score:  0.6691

 Confusion Matrix:
   [[395 206   2   0]
[ 71 175  25   1]
[  0  30  72   1]
[  0   0   4   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
   Accuracy:  0.6978
   Precision: 0.7144
   Recall:    0.6978
   F1 Score:  0.6977

 Confusion Matrix:
   [[509 103   5   0]
[317 615  94   8]
[  9  99 511  52]
[  0   0  38  39]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
   Accuracy:  0.7384
   Precision: 0.7654
   Recall:    0.7384
   F1 Score:  0.7411

 Confusion Matrix:
   [[391 185   4   0]
[ 37 513  59   1]
[  7  60 112   7]
[  0   0   0   0]]

 ── Eval 6: predict at 202202 → target 202210 ──
 Training samples:   13,860
 Test samples:       2,321  (areas in both 202202 and 202210)
 Training CatBoost...

 Results  (n_test=2,321):
   Accuracy:  0.7217
```

```
    Precision: 0.7185
    Recall:    0.7217
    F1 Score:  0.7197

  Confusion Matrix:
    [[641 198   2   0]
 [231 510  77   1]
 [  4  72 506  25]
 [  0   0  36  18]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.7309
    Precision: 0.7325
    Recall:    0.7309
    F1 Score:  0.7174

  Confusion Matrix:
    [[780  26   0   0]
 [291 581  44   1]
 [ 12 213 447   6]
 [  0  20  57  12]]

======================================================================
MODEL: CatBoost  |  FEATURE SET: GDELT only (no geo)
======================================================================
Using 154 features

  ── Eval 1: predict at 202310 → target 202406 ──
  Training samples:   11,873
  Test samples:       1,919  (areas in both 202310 and 202406)
  Training CatBoost...

  Results  (n_test=1,919):
    Accuracy:  0.3121
    Precision: 0.4287
    Recall:    0.3121
    F1 Score:  0.2548

  Confusion Matrix:
    [[410  32  30   8]
 [665  95  64   6]
 [326  43  84  15]
 [110   3  18  10]]

  ── Eval 2: predict at 202306 → target 202402 ──
  Training samples:   12,442
  Test samples:       2,313  (areas in both 202306 and 202402)
  Training CatBoost...

  Results  (n_test=2,313):
    Accuracy:  0.4419
```

```
  Precision: 0.4566
  Recall:    0.4419
  F1 Score:  0.3706

 Confusion Matrix:
   [[807  65  44   8]
[626 106  64  12]
[329  35  97  11]
[ 80   1  16  12]]

 ── Eval 3: predict at 202302 → target 202310 ──
 Training samples:   13,462
 Test samples:       982  (areas in both 202302 and 202310)
 Training CatBoost...

 Results  (n_test=982):
  Accuracy:  0.6008
  Precision: 0.5474
  Recall:    0.6008
  F1 Score:  0.5331

 Confusion Matrix:
   [[541  39  18   5]
[217  32  20   3]
[ 82   2  17   2]
[  4   0   0   0]]

 ── Eval 4: predict at 202210 → target 202306 ──
 Training samples:   13,865
 Test samples:       2,399  (areas in both 202210 and 202306)
 Training CatBoost...

 Results  (n_test=2,399):
  Accuracy:  0.2947
  Precision: 0.4246
  Recall:    0.2947
  F1 Score:  0.2160

 Confusion Matrix:
   [[545  33  34   5]
[876  79  58  21]
[553  32  67  19]
[ 52   1   8  16]]

 ── Eval 5: predict at 202206 → target 202302 ──
 Training samples:   13,794
 Test samples:       1,376  (areas in both 202206 and 202302)
 Training CatBoost...

 Results  (n_test=1,376):
  Accuracy:  0.4433
  Precision: 0.4713
  Recall:    0.4433
  F1 Score:  0.3539

 Confusion Matrix:
```

```
    [[522  34  17   7]
 [506  59  38   7]
 [136  18  29   3]
 [  0   0   0   0]]

  ── Eval 6: predict at 202202 → target 202210 ──
  Training samples:   13,860
  Test samples:       2,321  (areas in both 202202 and 202210)
  Training CatBoost...

  Results  (n_test=2,321):
    Accuracy:  0.3912
    Precision: 0.4230
    Recall:    0.3912
    F1 Score:  0.3067

  Confusion Matrix:
    [[743  50  42   6]
 [676  85  49   9]
 [480  40  72  15]
 [ 35   2   9   8]]

  ── Eval 7: predict at 202110 → target 202206 ──
  Training samples:   12,948
  Test samples:       2,490  (areas in both 202110 and 202206)
  Training CatBoost...

  Results  (n_test=2,490):
    Accuracy:  0.3514
    Precision: 0.4035
    Recall:    0.3514
    F1 Score:  0.2724

  Confusion Matrix:
    [[694  69  38   5]
 [775  96  44   2]
 [547  38  83  10]
 [ 62   9  16   2]]

======================================================================
SUMMARY: AVERAGE RESULTS BY MODEL & FEATURE SET
======================================================================

Logistic Regression — IPC only:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.7079
    Precision: 0.7325
    Recall:    0.7079
    F1 Score:  0.7065

  Per evaluation:
    202310→202406:  n=1,919  acc=0.613  f1=0.605
    202306→202402:  n=2,313  acc=0.745  f1=0.741
    202302→202310:  n=  982  acc=0.628  f1=0.646
    202210→202306:  n=2,399  acc=0.758  f1=0.756
```

```
    202206→202302:  n=1,376  acc=0.746  f1=0.747
    202202→202210:  n=2,321  acc=0.724  f1=0.721
    202110→202206:  n=2,490  acc=0.740  f1=0.729

Logistic Regression — IPC + GDELT base:
--------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.7053
    Precision: 0.7292
    Recall:    0.7053
    F1 Score:  0.7041

  Per evaluation:
    202310→202406:  n=1,919  acc=0.610  f1=0.603
    202306→202402:  n=2,313  acc=0.735  f1=0.730
    202302→202310:  n=  982  acc=0.623  f1=0.641
    202210→202306:  n=2,399  acc=0.759  f1=0.757
    202206→202302:  n=1,376  acc=0.750  f1=0.751
    202202→202210:  n=2,321  acc=0.723  f1=0.720
    202110→202206:  n=2,490  acc=0.737  f1=0.727

Logistic Regression — IPC + GDELT + temporal:
--------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.7049
    Precision: 0.7303
    Recall:    0.7049
    F1 Score:  0.7041

  Per evaluation:
    202310→202406:  n=1,919  acc=0.615  f1=0.608
    202306→202402:  n=2,313  acc=0.741  f1=0.736
    202302→202310:  n=  982  acc=0.613  f1=0.631
    202210→202306:  n=2,399  acc=0.757  f1=0.755
    202206→202302:  n=1,376  acc=0.747  f1=0.748
    202202→202210:  n=2,321  acc=0.724  f1=0.721
    202110→202206:  n=2,490  acc=0.739  f1=0.729

Logistic Regression — GDELT only:
--------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.7059
    Precision: 0.7344
    Recall:    0.7059
    F1 Score:  0.7049

  Per evaluation:
    202310→202406:  n=1,919  acc=0.614  f1=0.608
    202306→202402:  n=2,313  acc=0.773  f1=0.772
    202302→202310:  n=  982  acc=0.598  f1=0.615
    202210→202306:  n=2,399  acc=0.757  f1=0.756
    202206→202302:  n=1,376  acc=0.746  f1=0.747
    202202→202210:  n=2,321  acc=0.726  f1=0.722
    202110→202206:  n=2,490  acc=0.726  f1=0.715

Logistic Regression — IPC only (no geo):
```

```
--------------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6781
    Precision: 0.6818
    Recall:    0.6781
    F1 Score:  0.6666

  Per evaluation:
    202310→202406:  n=1,919  acc=0.508  f1=0.466
    202306→202402:  n=2,313  acc=0.667  f1=0.656
    202302→202310:  n=  982  acc=0.660  f1=0.674
    202210→202306:  n=2,399  acc=0.705  f1=0.691
    202206→202302:  n=1,376  acc=0.748  f1=0.748
    202202→202210:  n=2,321  acc=0.730  f1=0.720
    202110→202206:  n=2,490  acc=0.730  f1=0.711

Logistic Regression — IPC + GDELT base (no geo):
--------------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6772
    Precision: 0.7000
    Recall:    0.6772
    F1 Score:  0.6668

  Per evaluation:
    202310→202406:  n=1,919  acc=0.507  f1=0.467
    202306→202402:  n=2,313  acc=0.667  f1=0.658
    202302→202310:  n=  982  acc=0.661  f1=0.675
    202210→202306:  n=2,399  acc=0.705  f1=0.692
    202206→202302:  n=1,376  acc=0.745  f1=0.746
    202202→202210:  n=2,321  acc=0.729  f1=0.720
    202110→202206:  n=2,490  acc=0.727  f1=0.709

Logistic Regression — IPC + GDELT + temporal (no geo):
--------------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6755
    Precision: 0.7016
    Recall:    0.6755
    F1 Score:  0.6658

  Per evaluation:
    202310→202406:  n=1,919  acc=0.508  f1=0.471
    202306→202402:  n=2,313  acc=0.667  f1=0.660
    202302→202310:  n=  982  acc=0.657  f1=0.671
    202210→202306:  n=2,399  acc=0.704  f1=0.693
    202206→202302:  n=1,376  acc=0.740  f1=0.741
    202202→202210:  n=2,321  acc=0.726  f1=0.717
    202110→202206:  n=2,490  acc=0.726  f1=0.708

Logistic Regression — GDELT only (no geo):
--------------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.3505
    Precision: 0.4093
    Recall:    0.3505
```

```
    F1 Score:  0.2592

  Per evaluation:
    202310→202406:  n=1,919  acc=0.294  f1=0.228
    202306→202402:  n=2,313  acc=0.429  f1=0.351
    202302→202310:  n=  982  acc=0.281  f1=0.172
    202210→202306:  n=2,399  acc=0.287  f1=0.202
    202206→202302:  n=1,376  acc=0.440  f1=0.348
    202202→202210:  n=2,321  acc=0.380  f1=0.280
    202110→202206:  n=2,490  acc=0.343  f1=0.234

Random Forest — IPC only:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6343
    Precision: 0.7077
    Recall:    0.6343
    F1 Score:  0.6301

  Per evaluation:
    202310→202406:  n=1,919  acc=0.563  f1=0.535
    202306→202402:  n=2,313  acc=0.656  f1=0.666
    202302→202310:  n=  982  acc=0.461  f1=0.448
    202210→202306:  n=2,399  acc=0.664  f1=0.671
    202206→202302:  n=1,376  acc=0.730  f1=0.725
    202202→202210:  n=2,321  acc=0.670  f1=0.678
    202110→202206:  n=2,490  acc=0.696  f1=0.688

Random Forest — IPC + GDELT base:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6392
    Precision: 0.7012
    Recall:    0.6392
    F1 Score:  0.6390

  Per evaluation:
    202310→202406:  n=1,919  acc=0.576  f1=0.553
    202306→202402:  n=2,313  acc=0.658  f1=0.668
    202302→202310:  n=  982  acc=0.494  f1=0.502
    202210→202306:  n=2,399  acc=0.655  f1=0.661
    202206→202302:  n=1,376  acc=0.733  f1=0.733
    202202→202210:  n=2,321  acc=0.670  f1=0.678
    202110→202206:  n=2,490  acc=0.688  f1=0.678

Random Forest — IPC + GDELT + temporal:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6480
    Precision: 0.7045
    Recall:    0.6480
    F1 Score:  0.6487

  Per evaluation:
    202310→202406:  n=1,919  acc=0.582  f1=0.563
    202306→202402:  n=2,313  acc=0.663  f1=0.674
```

```
    202302→202310:  n=  982  acc=0.530  f1=0.540
    202210→202306:  n=2,399  acc=0.666  f1=0.672
    202206→202302:  n=1,376  acc=0.736  f1=0.735
    202202→202210:  n=2,321  acc=0.676  f1=0.684
    202110→202206:  n=2,490  acc=0.683  f1=0.673

Random Forest — GDELT only:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.5703
    Precision: 0.6383
    Recall:    0.5703
    F1 Score:  0.5594

  Per evaluation:
    202310→202406:  n=1,919  acc=0.547  f1=0.522
    202306→202402:  n=2,313  acc=0.613  f1=0.615
    202302→202310:  n=  982  acc=0.412  f1=0.378
    202210→202306:  n=2,399  acc=0.597  f1=0.606
    202206→202302:  n=1,376  acc=0.545  f1=0.513
    202202→202210:  n=2,321  acc=0.629  f1=0.639
    202110→202206:  n=2,490  acc=0.649  f1=0.643

Random Forest — IPC only (no geo):
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6795
    Precision: 0.7054
    Recall:    0.6795
    F1 Score:  0.6768

  Per evaluation:
    202310→202406:  n=1,919  acc=0.522  f1=0.500
    202306→202402:  n=2,313  acc=0.667  f1=0.673
    202302→202310:  n=  982  acc=0.660  f1=0.674
    202210→202306:  n=2,399  acc=0.701  f1=0.700
    202206→202302:  n=1,376  acc=0.744  f1=0.746
    202202→202210:  n=2,321  acc=0.728  f1=0.725
    202110→202206:  n=2,490  acc=0.734  f1=0.720

Random Forest — IPC + GDELT base (no geo):
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6745
    Precision: 0.7012
    Recall:    0.6745
    F1 Score:  0.6723

  Per evaluation:
    202310→202406:  n=1,919  acc=0.522  f1=0.502
    202306→202402:  n=2,313  acc=0.665  f1=0.669
    202302→202310:  n=  982  acc=0.650  f1=0.665
    202210→202306:  n=2,399  acc=0.698  f1=0.698
    202206→202302:  n=1,376  acc=0.738  f1=0.739
    202202→202210:  n=2,321  acc=0.726  f1=0.723
    202110→202206:  n=2,490  acc=0.724  f1=0.710
```

```
Random Forest — IPC + GDELT + temporal (no geo):
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6720
    Precision: 0.6971
    Recall:    0.6720
    F1 Score:  0.6694

  Per evaluation:
    202310→202406:  n=1,919  acc=0.519  f1=0.498
    202306→202402:  n=2,313  acc=0.661  f1=0.665
    202302→202310:  n=  982  acc=0.645  f1=0.660
    202210→202306:  n=2,399  acc=0.697  f1=0.696
    202206→202302:  n=1,376  acc=0.737  f1=0.739
    202202→202210:  n=2,321  acc=0.720  f1=0.716
    202110→202206:  n=2,490  acc=0.726  f1=0.712

Random Forest — GDELT only (no geo):
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.3997
    Precision: 0.4366
    Recall:    0.3997
    F1 Score:  0.3206

  Per evaluation:
    202310→202406:  n=1,919  acc=0.316  f1=0.264
    202306→202402:  n=2,313  acc=0.437  f1=0.367
    202302→202310:  n=  982  acc=0.585  f1=0.509
    202210→202306:  n=2,399  acc=0.294  f1=0.216
    202206→202302:  n=1,376  acc=0.434  f1=0.336
    202202→202210:  n=2,321  acc=0.388  f1=0.295
    202110→202206:  n=2,490  acc=0.344  f1=0.258

CatBoost — IPC only:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6725
    Precision: 0.7088
    Recall:    0.6725
    F1 Score:  0.6741

  Per evaluation:
    202310→202406:  n=1,919  acc=0.539  f1=0.519
    202306→202402:  n=2,313  acc=0.674  f1=0.685
    202302→202310:  n=  982  acc=0.643  f1=0.660
    202210→202306:  n=2,399  acc=0.689  f1=0.692
    202206→202302:  n=1,376  acc=0.743  f1=0.745
    202202→202210:  n=2,321  acc=0.695  f1=0.701
    202110→202206:  n=2,490  acc=0.725  f1=0.717

CatBoost — IPC + GDELT base:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6734
```

```
    Precision: 0.7112
    Recall:    0.6734
    F1 Score:  0.6754

  Per evaluation:
    202310→202406:  n=1,919  acc=0.539  f1=0.519
    202306→202402:  n=2,313  acc=0.671  f1=0.682
    202302→202310:  n=  982  acc=0.644  f1=0.661
    202210→202306:  n=2,399  acc=0.697  f1=0.701
    202206→202302:  n=1,376  acc=0.738  f1=0.742
    202202→202210:  n=2,321  acc=0.694  f1=0.700
    202110→202206:  n=2,490  acc=0.730  f1=0.721

CatBoost — IPC + GDELT + temporal:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6742
    Precision: 0.7102
    Recall:    0.6742
    F1 Score:  0.6760

  Per evaluation:
    202310→202406:  n=1,919  acc=0.545  f1=0.526
    202306→202402:  n=2,313  acc=0.680  f1=0.690
    202302→202310:  n=  982  acc=0.635  f1=0.653
    202210→202306:  n=2,399  acc=0.696  f1=0.700
    202206→202302:  n=1,376  acc=0.740  f1=0.742
    202202→202210:  n=2,321  acc=0.696  f1=0.702
    202110→202206:  n=2,490  acc=0.727  f1=0.719

CatBoost — GDELT only:
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6268
    Precision: 0.6757
    Recall:    0.6268
    F1 Score:  0.6264

  Per evaluation:
    202310→202406:  n=1,919  acc=0.588  f1=0.569
    202306→202402:  n=2,313  acc=0.658  f1=0.662
    202302→202310:  n=  982  acc=0.511  f1=0.529
    202210→202306:  n=2,399  acc=0.665  f1=0.672
    202206→202302:  n=1,376  acc=0.669  f1=0.663
    202202→202210:  n=2,321  acc=0.647  f1=0.649
    202110→202206:  n=2,490  acc=0.650  f1=0.640

CatBoost — IPC only (no geo):
----------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6795
    Precision: 0.7054
    Recall:    0.6795
    F1 Score:  0.6768

  Per evaluation:
```

```
    202310→202406:  n=1,919  acc=0.522  f1=0.500
    202306→202402:  n=2,313  acc=0.667  f1=0.673
    202302→202310:  n=  982  acc=0.660  f1=0.674
    202210→202306:  n=2,399  acc=0.701  f1=0.700
    202206→202302:  n=1,376  acc=0.744  f1=0.746
    202202→202210:  n=2,321  acc=0.728  f1=0.725
    202110→202206:  n=2,490  acc=0.734  f1=0.720

CatBoost — IPC + GDELT base (no geo):
------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6782
    Precision: 0.7037
    Recall:    0.6782
    F1 Score:  0.6756

  Per evaluation:
    202310→202406:  n=1,919  acc=0.521  f1=0.499
    202306→202402:  n=2,313  acc=0.668  f1=0.673
    202302→202310:  n=  982  acc=0.659  f1=0.673
    202210→202306:  n=2,399  acc=0.700  f1=0.700
    202206→202302:  n=1,376  acc=0.743  f1=0.745
    202202→202210:  n=2,321  acc=0.726  f1=0.724
    202110→202206:  n=2,490  acc=0.730  f1=0.716

CatBoost — IPC + GDELT + temporal (no geo):
------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.6758
    Precision: 0.7005
    Recall:    0.6758
    F1 Score:  0.6737

  Per evaluation:
    202310→202406:  n=1,919  acc=0.519  f1=0.496
    202306→202402:  n=2,313  acc=0.669  f1=0.674
    202302→202310:  n=  982  acc=0.654  f1=0.669
    202210→202306:  n=2,399  acc=0.698  f1=0.698
    202206→202302:  n=1,376  acc=0.738  f1=0.741
    202202→202210:  n=2,321  acc=0.722  f1=0.720
    202110→202206:  n=2,490  acc=0.731  f1=0.717

CatBoost — GDELT only (no geo):
------------------------------------------------------------------------
  Average Metrics (across 7 assessments, 13,800 total test samples):
    Accuracy:  0.4051
    Precision: 0.4508
    Recall:    0.4051
    F1 Score:  0.3296

  Per evaluation:
    202310→202406:  n=1,919  acc=0.312  f1=0.255
    202306→202402:  n=2,313  acc=0.442  f1=0.371
    202302→202310:  n=  982  acc=0.601  f1=0.533
    202210→202306:  n=2,399  acc=0.295  f1=0.216
    202206→202302:  n=1,376  acc=0.443  f1=0.354
```

```
    202202→202210:  n=2,321  acc=0.391  f1=0.307
    202110→202206:  n=2,490  acc=0.351  f1=0.272

======================================================================
SUMMARY TABLE
======================================================================
              Model                     Feature Set  N Features  IPC  GDELT
Base  GDELT Temporal  Geographic  Accuracy  Precision   Recall  F1 Score  To
tal Test  N Evals
Logistic Regression                       IPC only        3332    1
0               0        3331  0.707928   0.732537 0.707928  0.706539
13800        7
Logistic Regression                     GDELT only        3485    0
119              35        3331  0.705925   0.734445 0.705925  0.704864
13800        7
Logistic Regression         IPC + GDELT + temporal        3486    1
119              35        3331  0.704939   0.730285 0.704939  0.704090
13800        7
Logistic Regression               IPC + GDELT base        3451    1
119               0        3331  0.705277   0.729226 0.705277  0.704050
13800        7
          CatBoost              IPC only (no geo)           1    1
0                0           0  0.679529   0.705431 0.679529  0.676829
13800        7
      Random Forest              IPC only (no geo)           1    1
0                0           0  0.679529   0.705431 0.679529  0.676829
13800        7
          CatBoost          IPC + GDELT + temporal        3486    1
119              35        3331  0.674180   0.710209 0.674180  0.675994
13800        7
          CatBoost       IPC + GDELT base (no geo)         120    1
119               0           0  0.678150   0.703716 0.678150  0.675591
13800        7
          CatBoost               IPC + GDELT base        3451    1
119               0        3331  0.673354   0.711218 0.673354  0.675350
13800        7
          CatBoost                       IPC only        3332    1
0                0        3331  0.672506   0.708834 0.672506  0.674136
13800        7
          CatBoost IPC + GDELT + temporal (no geo)         155    1
119              35           0  0.675830   0.700528 0.675830  0.673673
13800        7
      Random Forest       IPC + GDELT base (no geo)         120    1
119               0           0  0.674481   0.701159 0.674481  0.672265
13800        7
      Random Forest IPC + GDELT + temporal (no geo)         155    1
119              35           0  0.672026   0.697146 0.672026  0.669416
13800        7
Logistic Regression       IPC + GDELT base (no geo)         120    1
119               0           0  0.677242   0.700024 0.677242  0.666782
13800        7
Logistic Regression              IPC only (no geo)           1    1
0                0           0  0.678111   0.681797 0.678111  0.666570
13800        7
Logistic Regression IPC + GDELT + temporal (no geo)         155    1
119              35           0  0.675481   0.701638 0.675481  0.665792
```

```
13800        7
      Random Forest          IPC + GDELT + temporal        3486    1
119              35        3331  0.647966   0.704533 0.647966  0.648662
13800        7
      Random Forest                IPC + GDELT base        3451    1
119               0        3331  0.639173   0.701183 0.639173  0.639013
13800        7
      Random Forest                        IPC only        3332    1
0                0        3331  0.634269   0.707684 0.634269  0.630074
13800        7
           CatBoost                       GDELT only        3485    0
119              35        3331  0.626788   0.675695 0.626788  0.626399
13800        7
      Random Forest                       GDELT only        3485    0
119              35        3331  0.570304   0.638342 0.570304  0.559448
13800        7
           CatBoost             GDELT only (no geo)         154    0
119              35           0  0.405063   0.450758 0.405063  0.329641
13800        7
      Random Forest             GDELT only (no geo)         154    0
119              35           0  0.399662   0.436594 0.399662  0.320561
13800        7
Logistic Regression             GDELT only (no geo)         154    0
119              35           0  0.350527   0.409250 0.350527  0.259224
13800        7

======================================================================
RANDOM FOREST: AVERAGE FEATURE IMPORTANCES
======================================================================

── IPC only (averaged over 7 splits) ──
Feature                                                    Importance
----------------------------------------------------------------------
  IPC_last                                                     0.2187
  ADMIN0_yemen                                                 0.0845
  ADMIN0_somalia                                               0.0460
  ADMIN0_nigeria                                               0.0452
  ADMIN0_south sudan                                           0.0412
  ADMIN0_ethiopia                                              0.0392
  ADMIN0_sudan                                                 0.0321
  ADMIN1_tigray                                                0.0248
  ADMIN0_burundi                                               0.0198
  ADMIN0_kenya                                                 0.0177
  ADMIN0_uganda                                                0.0145
  ADMIN1_mudug                                                 0.0122
  ADMIN0_mozambique                                            0.0102
  ADMIN1_hajjah                                                0.0101
  ADMIN0_zimbabwe                                              0.0100
  ADMIN1_hadramaut                                             0.0095
  ADMIN2_south tigray                                          0.0086
  ADMIN1_somali                                                0.0081
  ADMIN2_juba                                                  0.0080
  ADMIN1_borno                                                 0.0078
  ADMIN2_hobyo                                                 0.0070
  ADMIN1_central equatoria                                     0.0069
  ADMIN1_marib                                                 0.0067
```

Feature	Importance
ADMIN0_democratic republic of the congo	0.0063
ADMIN1_amran	0.0063
... (3307 more features omitted)	

── IPC + GDELT base (averaged over 7 splits) ──

Feature	Importance
IPC_last	0.1739
ADMIN0_yemen	0.0786
ADMIN0_nigeria	0.0409
ADMIN0_somalia	0.0403
ADMIN0_south sudan	0.0337
ADMIN0_ethiopia	0.0271
ADMIN0_sudan	0.0257
ADMIN0_burundi	0.0158
ADMIN0_kenya	0.0157
ADMIN0_uganda	0.0106
ADMIN1_borno	0.0103
ADMIN1_hajjah	0.0101
ADMIN1_tigray	0.0101
ADMIN1_somali	0.0092
ADMIN0_mozambique	0.0081
ADMIN1_hadramaut	0.0075
ADMIN0_zimbabwe	0.0071
ADMIN1_amran	0.0069
ADMIN1_afar	0.0060
ADMIN1_marib	0.0059
ADMIN1_al hudaydah	0.0056
ADMIN0_democratic republic of the congo	0.0055
ADMIN1_western	0.0048
ADMIN1_turkana	0.0048
ADMIN1_south darfur	0.0047
... (3426 more features omitted)	

── IPC + GDELT + temporal (averaged over 7 splits) ──

Feature	Importance
IPC_last	0.1802
ADMIN0_yemen	0.0699
ADMIN0_nigeria	0.0383
ADMIN0_somalia	0.0338
ADMIN0_south sudan	0.0305
ADMIN0_sudan	0.0244
ADMIN0_ethiopia	0.0222
ADMIN0_kenya	0.0147
ADMIN0_burundi	0.0145
evt_displaced_freq_mean_rolling_6	0.0105
ADMIN0_uganda	0.0097
ADMIN1_hajjah	0.0092
ADMIN1_borno	0.0084
ADMIN1_somali	0.0084
ADMIN1_tigray	0.0075
ADMIN0_mozambique	0.0074
evt_displaced_freq_mean_rolling_3	0.0069
ADMIN1_hadramaut	0.0069
ADMIN1_amran	0.0066

```
  evt_compound_score_mean_rolling_6                          0.0066
  evt_sentiment.compound_mean_rolling_6                      0.0062
  ADMIN1_afar                                                0.0059
  ADMIN0_zimbabwe                                            0.0057
  ADMIN1_marib                                               0.0054
  evt_food_insecurity_freq_mean_rolling_6                    0.0053
  ... (3461 more features omitted)

── GDELT only (averaged over 7 splits) ──
Feature                                                   Importance
------------------------------------------------------------------------
  ADMIN0_yemen                                               0.0833
  ADMIN0_nigeria                                             0.0459
  ADMIN0_somalia                                             0.0437
  ADMIN0_south sudan                                         0.0394
  ADMIN0_sudan                                               0.0278
  ADMIN0_ethiopia                                            0.0269
  ADMIN0_burundi                                             0.0190
  ADMIN0_kenya                                               0.0181
  evt_displaced_freq_mean_rolling_6                          0.0134
  ADMIN0_uganda                                              0.0121
  ADMIN1_somali                                              0.0114
  ADMIN1_hajjah                                              0.0113
  ADMIN1_borno                                               0.0108
  ADMIN1_tigray                                              0.0092
  ADMIN0_mozambique                                          0.0086
  ADMIN1_hadramaut                                           0.0085
  evt_displaced_freq_mean_rolling_3                          0.0082
  ADMIN1_amran                                               0.0081
  evt_sentiment.compound_mean_rolling_6                      0.0080
  evt_compound_score_mean_rolling_6                          0.0079
  ADMIN1_afar                                                0.0072
  ADMIN0_zimbabwe                                            0.0068
  ADMIN1_al hudaydah                                         0.0068
  ADMIN1_marib                                               0.0065
  evt_food_insecurity_freq_mean_rolling_6                    0.0061
  ... (3460 more features omitted)

── IPC only (no geo) (averaged over 7 splits) ──
Feature                                                   Importance
------------------------------------------------------------------------
  IPC_last                                                   1.0000

── IPC + GDELT base (no geo) (averaged over 7 splits) ──
Feature                                                   Importance
------------------------------------------------------------------------
  IPC_last                                                   0.6200
  evt_sentiment.compound_sum                                 0.0113
  evt_displaced_freq_mean                                    0.0111
  evt_sentiment.neg_mean                                     0.0111
  evt_neg_score_mean                                         0.0110
  evt_compound_score_sum                                     0.0109
  evt_displaced_count_mean                                   0.0096
  evt_neu_score_mean                                         0.0094
  evt_compound_score_mean                                    0.0093
  evt_sentiment.neu_mean                                     0.0093
```

evt_sentiment.pos_sum	0.0090
evt_neu_score_sum	0.0090
evt_neg_score_sum	0.0089
evt_sentiment.neg_sum	0.0088
evt_sentiment.compound_mean	0.0087
evt_sentiment.neu_sum	0.0085
evt_pos_score_mean	0.0083
evt_sentiment.pos_mean	0.0083
evt_pos_score_sum	0.0081
evt_fatalities_count_mean	0.0072
evt_fatalities_count_sum	0.0071
evt_pred_resource_cash_aid_mean	0.0065
evt_displaced_count_sum	0.0063
evt_fatalities_freq_mean	0.0060
evt_displaced_freq_sum	0.0058
... (95 more features omitted)	

── IPC + GDELT + temporal (no geo) (averaged over 7 splits) ──

Feature	Importance
IPC_last	0.4736
evt_displaced_freq_mean_rolling_6	0.0300
evt_sentiment.compound_mean_rolling_6	0.0218
evt_compound_score_mean_rolling_6	0.0216
evt_fatalities_freq_mean_rolling_6	0.0180
evt_displaced_freq_mean_rolling_3	0.0162
evt_sentiment.compound_mean_rolling_3	0.0150
evt_compound_score_mean_rolling_3	0.0147
evt_food_insecurity_freq_mean_rolling_6	0.0140
evt_agriculture_freq_mean_rolling_6	0.0113
evt_fatalities_freq_mean_rolling_3	0.0103
evt_sentiment.compound_mean_lag1	0.0094
evt_displaced_freq_mean_lag1	0.0093
evt_compound_score_mean_lag1	0.0092
evt_compound_score_mean_lag2	0.0086
evt_sentiment.compound_mean_lag2	0.0085
evt_displaced_freq_mean_lag2	0.0077
evt_sentiment.neg_mean	0.0072
evt_displaced_freq_mean	0.0072
evt_sentiment.compound_sum	0.0071
evt_compound_score_sum	0.0071
evt_neg_score_mean	0.0070
evt_fatalities_freq_mean_lag2	0.0067
evt_food_insecurity_freq_mean_rolling_3	0.0064
evt_neu_score_mean	0.0063
... (130 more features omitted)	

── GDELT only (no geo) (averaged over 7 splits) ──

Feature	Importance
evt_displaced_freq_mean_rolling_6	0.0561
evt_sentiment.compound_mean_rolling_6	0.0441
evt_compound_score_mean_rolling_6	0.0432
evt_fatalities_freq_mean_rolling_6	0.0345
evt_displaced_freq_mean_rolling_3	0.0326
evt_sentiment.compound_mean_rolling_3	0.0280

```
evt_compound_score_mean_rolling_3                    0.0276
evt_food_insecurity_freq_mean_rolling_6              0.0275
evt_agriculture_freq_mean_rolling_6                  0.0216
evt_fatalities_freq_mean_rolling_3                   0.0196
evt_compound_score_mean_lag1                         0.0182
evt_sentiment.compound_mean_lag1                     0.0181
evt_displaced_freq_mean_lag2                         0.0169
evt_sentiment.compound_mean_lag2                     0.0167
evt_displaced_freq_mean_lag1                         0.0161
evt_compound_score_mean_lag2                         0.0161
evt_neg_score_mean                                   0.0131
evt_displaced_freq_mean                              0.0129
evt_sentiment.neg_mean                               0.0127
evt_sentiment.compound_sum                           0.0127
evt_fatalities_freq_mean_lag2                        0.0127
evt_neu_score_mean                                   0.0123
evt_sentiment.neu_mean                               0.0122
evt_compound_score_sum                               0.0121
evt_fatalities_freq_mean_lag1                        0.0117
... (129 more features omitted)
```

Why Literature Shows Better Results: Key Differences

Potential Reasons for Better Performance in Literature:

1. **Different Prediction Tasks**:
   - **Transitions/Changes**: Predicting when CS_score will worsen (crisis onset) rather than exact level
   - **Early Warning**: Predicting 2-4 periods ahead (forecasting) rather than next period (nowcasting)
   - **Binary Classification**: Predicting crisis (CS≥3) vs non-crisis (CS<3) rather than 5-class classification
2. **Feature Engineering**:
   - **Temporal Patterns**: Using lagged and rolling features (as we just added) rather than only current-period values
   - **Anomaly Detection**: Focusing on deviations from historical patterns
   - **Interaction Features**: Combining GDELT with other data sources (weather, prices, etc.)
3. **Data Coverage**:
   - **Spatial Aggregation**: Some studies aggregate to ADMIN0 or ADMIN1 level where coverage is better
   - **Temporal Aggregation**: Using quarterly rather than monthly data

- **Filtering**: Focusing on regions/periods with good GDELT coverage

4. **Evaluation Metrics**:
    - **Recall for Rare Events**: Focusing on detecting crises (high recall for CS≥3) rather than overall accuracy
    - **Early Detection**: Measuring how early they detect transitions, not just accuracy
5. **Baseline Comparison**:
    - **Weaker Baselines**: Some studies don't include `previous_CS` in baseline, making improvement easier to show
    - **Different Baselines**: Comparing against simpler models or using different evaluation windows

Our Current Approach:

- ✅ Now includes temporal GDELT features (lags, rolling windows, anomalies, escalation)
- ✅ Uses same framework as previous work (allowing fair comparison)
- ⚠️ Still predicting exact CS_score level (highly autocorrelated)
- ⚠️ Using monthly data with CS_score available every 4 months

Potential Improvements:

1. **Predict Transitions**: Predict if CS_score will worsen (CS_t+1 > CS_t) rather than exact value
2. **Forecasting Horizon**: Predict 2-4 periods ahead instead of next period
3. **Crisis Detection**: Binary classification (crisis vs non-crisis) with focus on recall
4. **Feature Selection**: Use only temporal GDELT features, filter out low-importance current-period features

Feature Engineering Summary

Features Created:

1. **List Aggregations** (from events):
    - Mean, Max, Sum, Count for all list features
    - Prefixed with `evt_`
2. **Derived Features**:
    - Crisis severity indicators
    - Coverage intensity metrics
3. **Temporal Features**:

- Lag features (CS_score_lag1, lag2, lag3)
- Moving averages (ma2, ma3)
- Change and percentage change features
- Cyclical period encoding (sin/cos)

4. **Geographic Features**:

- Aggregated CS_score by ADMIN0 and ADMIN1 levels
- Standard deviations by geographic level
- Count of regions with same score

5. **Interaction Features**:

- Ratios (casualty_rate, aid_per_casualty)
- Normalized features (tone_abs_normalized)
- Coverage metrics (mentions_per_source, articles_per_source)

Next Steps for ML:

1. **Feature Selection**: Consider using:

- Correlation-based selection
- Mutual information
- Recursive feature elimination
- L1 regularization (Lasso)

2. **Categorical Encoding**: If you have categorical features:

- One-hot encoding for low cardinality
- Target encoding for high cardinality
- Embedding for very high cardinality

3. **Scaling**: Consider:

- StandardScaler or MinMaxScaler for numeric features
- Especially important for distance-based algorithms

4. **Model Suggestions**:

- Random Forest (handles non-linear relationships well)
- Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Neural Networks (if you have enough data)
- Consider class weights if classes are imbalanced